

# MAMBA-Viper

## Specification for a Key Encapsulation Mechanism from Module Learning with Quantization

---

Shanxiang Lyu<sup>1\*</sup>   Ke Ma<sup>1</sup>   Jiarui Deng<sup>1</sup>  
Junzuo Lai<sup>1\*</sup>   Ling Liu<sup>2</sup>   Xianhui Lu<sup>3</sup>

<sup>1</sup> College of Cyber Security, Jinan University, Guangzhou, China

<sup>2</sup> Guangzhou Institute of Technology, Xidian University, Guangzhou, China

<sup>3</sup> Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China

\* Correspondence: [lsx07@jnu.edu.cn](mailto:lsx07@jnu.edu.cn), [laijunzuo@gmail.com](mailto:laijunzuo@gmail.com)

APRIL 28, 2026

---

### Abstract

MAMBA-Viper is a family of key encapsulation mechanisms (KEMs) based on the Module Learning with Quantization (MLWQ) problem. It uses public dithered coefficient-wise quantization as a native cryptographic operation rather than as an auxiliary compression layer. In the divisible power-of-two setting, the transmitted split representation admits a public compensated normal form with an explicit bounded quantization-error distribution. The submitted parameter sets use the modulus  $q = 2^{12}$ , or  $q = 2^{13}$  for the two highest-security profiles, together with ring dimension  $n = 256$  and compression moduli dividing  $q$ . The default arithmetic path uses Toom–Cook/Karatsuba multiplication over  $R_q$ , so the scheme is instantiated entirely with native power-of-two ring arithmetic.

This document provides a standardization-oriented specification of the algorithms, parameters, correctness analysis, concrete security estimates, known attack surfaces, implementation requirements, and conformance criteria for MAMBA-Viper. All concrete parameters, byte sizes, and benchmark numbers are derived from the reference and optimized implementations.

---

# Contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>4</b>  |
| 1.1      | Design Rationale . . . . .                            | 5         |
| <b>2</b> | <b>General Algorithm Specification</b>                | <b>6</b>  |
| 2.1      | Notation . . . . .                                    | 6         |
| 2.2      | Message Encoding and Decoding . . . . .               | 6         |
| 2.3      | Specification of Viper.PKE . . . . .                  | 7         |
| 2.4      | Specification of Viper.KEM . . . . .                  | 9         |
| <b>3</b> | <b>Parameter Settings</b>                             | <b>10</b> |
| 3.1      | Selected Parameter Sets . . . . .                     | 10        |
| 3.2      | Space Overheads and External Byte Lengths . . . . .   | 11        |
| 3.3      | Decryption Correctness and Security Summary . . . . . | 11        |
| <b>4</b> | <b>Performance Evaluation</b>                         | <b>12</b> |
| 4.1      | Experimental Setup and Platforms . . . . .            | 12        |
| 4.2      | Implementation Variants . . . . .                     | 12        |
| 4.3      | x86 Implementation Results . . . . .                  | 13        |
| 4.4      | Cortex-M4 Implementation Results . . . . .            | 13        |
| 4.5      | Comparison to Other Lattice-Based KEMs . . . . .      | 13        |
| <b>5</b> | <b>Known Answer Tests</b>                             | <b>16</b> |
| <b>6</b> | <b>Security Strength</b>                              | <b>17</b> |
| 6.1      | Security Definitions and Claims . . . . .             | 17        |
| 6.2      | Correctness of the PKE Scheme . . . . .               | 18        |
| 6.3      | Theoretical Security . . . . .                        | 19        |
| 6.4      | Concrete Security . . . . .                           | 20        |
| 6.5      | Side-Channel Security Considerations . . . . .        | 20        |
| <b>7</b> | <b>Advantages and Limitations</b>                     | <b>21</b> |
| 7.1      | Advantages . . . . .                                  | 21        |
| 7.2      | Limitations . . . . .                                 | 22        |
| <b>8</b> | <b>Implementation Specification</b>                   | <b>23</b> |
| 8.1      | External API and Implementation Boundary . . . . .    | 23        |
| 8.2      | Integer Representation . . . . .                      | 24        |
| 8.3      | Serialized Objects . . . . .                          | 24        |
| 8.4      | Bit Packing and Parsing . . . . .                     | 24        |
| 8.5      | Seed Expansion and Public Dithers . . . . .           | 25        |
| 8.6      | Secret Sampling . . . . .                             | 25        |
| 8.7      | Dithered Quantization and Compensation . . . . .      | 25        |
| 8.8      | $E_8$ Labeling and Delabeling . . . . .               | 26        |
| 8.9      | Polynomial Arithmetic . . . . .                       | 27        |
| 8.10     | Constant-Time Decapsulation . . . . .                 | 27        |
| 8.11     | Validation Requirements . . . . .                     | 27        |
| <b>A</b> | <b>Mathematical Foundations</b>                       | <b>28</b> |
| <b>B</b> | <b>Decryption Failure Analysis</b>                    | <b>29</b> |

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>C</b> | <b>Security Proof Details</b>                               | <b>31</b> |
| C.1      | IND-CPA Security . . . . .                                  | 31        |
| C.2      | IND-CCA Security . . . . .                                  | 33        |
| <b>D</b> | <b>Additional Benchmark Results</b>                         | <b>34</b> |
| D.1      | Native x86 Comparison to Other Lattice-Based KEMs . . . . . | 34        |
| D.2      | Detailed Native x86 Breakdown . . . . .                     | 34        |

# 1 Introduction

**Viper** (formally designated as **MAMBA-Viper**) is a lattice-based key encapsulation mechanism (KEM) engineered for post-quantum cryptography standardization. The underlying scheme is built upon the Module Learning with Quantization (MLWQ) problem [LLL24]—a modern hardness assumption that natively bridges the compact communication performance of Module Learning with Rounding (MLWR) with the rigorous, transparent security reductions of Module Learning with Errors (MLWE).

In contrast to MLWE-based primitives such as ML-KEM, **Viper** natively integrates quantization into its core design as a compression mechanism, thereby achieving substantially smaller public keys and ciphertexts at equivalent security levels. Concurrently, unlike MLWR-based designs such as Saber, the quantization noise in **Viper** remains statistically independent of secret-dependent quantities. This structural independence enables a direct security reduction to MLWE without relying on heuristic noise-merging or approximation arguments.

The protocol targets indistinguishability under adaptive chosen-ciphertext attacks (IND-CCA) in both the classical random-oracle model (ROM) and the quantum random-oracle model (QROM). Its cryptographic soundness is established via a tight reduction from the search version of the MLWQ problem, which maps directly to the standard MLWE problem under an explicit error distribution.

**Viper** features a highly portable reference implementation in standard C, an optimized implementation leveraging AVX2 vector instructions, and a dedicated Cortex-M4 port for embedded performance evaluation. All implementations are structurally constant-time and have been formally verified for correctness against comprehensive known-answer tests (KATs). To accommodate diverse architectural constraints and deployment requirements, **Viper** defines five distinct parameter sets covering nominal security targets of 128, 192, 256, 384, and 512 bits.

Table 1 summarizes the high-level structural distinctions between **Viper**, ML-KEM, and Saber. A comprehensive executive summary detailing the concrete bandwidth overheads and bit-security margins across all proposed **Viper** profiles is provided in Table 2.

Table 1: High-level architectural comparison of **Viper** with ML-KEM and Saber.

| Feature                | ML-KEM            | Saber                     | Viper (Ours)                     |
|------------------------|-------------------|---------------------------|----------------------------------|
| Underlying Problem     | MLWE              | MLWR                      | MLWQ                             |
| Modulus Arithmetic     | Prime (3329)      | Power-of-two ( $2^{13}$ ) | Power-of-two ( $2^{12}/2^{13}$ ) |
| Error Source           | Explicit Sampling | Deterministic Rounding    | Public Dithered Quantization     |
| Noise Independence     | Yes               | No                        | Yes                              |
| Error Correction Layer | Scalar Rounding   | Scalar Rounding           | $E_8$ Lattice Code               |
| Bandwidth Compression  | Rounding          | Inherent from Rounding    | Inherent from Quantization       |

Table 2: Executive summary of **Viper** profiles: bandwidth and concrete security margins.

| Scheme    | pk  (B) | ct  (B) | sk  (B) | MATZOV (C) | Core-SVP (C) | Core-SVP (Q) |
|-----------|---------|---------|---------|------------|--------------|--------------|
| Viper-128 | 608     | 736     | 1424    | 141.9      | 118.6        | 108.7        |
| Viper-192 | 992     | 1088    | 2200    | 199.4      | 177.5        | 162.1        |
| Viper-256 | 1312    | 1472    | 2912    | 266.0      | 246.2        | 225.3        |
| Viper-384 | 2496    | 2656    | 5488    | 410.2      | 393.7        | 361.1        |
| Viper-512 | 3200    | 3456    | 7040    | 564.6      | 549.5        | 521.0        |

## 1.1 Design Rationale

This section justifies the foundational algebraic design choices, performance–security trade-offs, and mathematical rationales driving the architecture of *Viper*.

### 1.1.1 MLWQ as the Security Primitive

The contemporary post-quantum lattice KEM landscape is heavily influenced by two pioneering paradigms, each exposing a distinct structural or theoretical trade-off:

- **ML-KEM** [BDK<sup>+</sup>18] relies on explicit error sampling and lacks native quantization within its underlying primitive, necessitating auxiliary polynomial compression that introduces additional bandwidth overhead.
- **Saber** [DKRV18] leverages deterministic learning with rounding (MLWR) [BPR12] to achieve compact communication artifacts. However, its deterministic rounding error is inherently dependent on the secret-dependent quantity **As**. This dependency thwarts direct, tight security reductions and requires heuristic non-trivial approximations.

By deploying the MLWQ formulation, *Viper* bridges this theoretical gap, simultaneously capturing the communication-compacting properties of MLWR alongside the rigorous reducibility of MLWE. The central enabling mechanism is public dithered quantization [LLL24, Zam14]. By introducing a pseudo-random, publicly accessible dither vector prior to quantization and subtracting it during reconstruction, the resulting quantization noise becomes statistically independent of the hidden input data. This exact algebraic independence enables a tight, non-heuristic security reduction directly to standard MLWE.

### 1.1.2 Power-of-Two Arithmetic and Implementation Efficiency

*Viper* adopts power-of-two modulus arithmetic, setting  $q = 2^{12}$  for the *Viper*-128, *Viper*-192, and *Viper*-256 profiles, and  $q = 2^{13}$  for the *Viper*-384 and *Viper*-512 profiles. This choice is central to the design: it yields exceptional simplicity in coefficient-wise quantization, reconstruction, and modular reduction.

Most notably, coefficient-wise quantization is reducible to simple bitwise additions, shifts, and masks, since all compression moduli divide  $q$ . Consequently, expanding split quantization labels into their compensated normal forms requires only fixed-latency integer operations. Because all coefficient operations fit naturally into unsigned fixed-width integer types, *Viper* eliminates variable-time modular reduction, modular inversion, and rejection-based arithmetic in its primary coefficient layer. This minimizes implementation complexity and facilitates constant-time execution.

The primary architectural trade-off is that a power-of-two modulus does not provide a native NTT-friendly field for negacyclic multiplication over the primary ring  $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ . The submitted optimized `API_PKC` implementation therefore deploys an AVX2 vectorized backend utilizing Toom–Cook and Karatsuba multiplication by default. The source tree also contains an auxiliary-prime NTT backend based on NTT-friendly companion moduli and CRT reconstruction techniques [CHK<sup>+</sup>21, ACC<sup>+</sup>22]. This auxiliary backend is an optional implementation alternative and is not enabled by the default submitted `API_PKC` build. Thus, the scheme semantics, byte encodings, KAT outputs, and formal API behavior remain strictly decoupled from the internal multiplication backend.

### 1.1.3 Lattice Coding for Error Correction

To achieve high noise tolerance with minimal bandwidth overhead, *Viper* employs an  $E_8$  root lattice (the Gosset lattice) as its message coding layer. The  $E_8$  lattice is the unique even unimodular lattice in eight dimensions, renowned for its exceptionally dense sphere packing. This geometric property enables highly efficient nearest-lattice decoding and strong error-correction capability at moderate computational complexity.

Although higher-dimensional lattices, such as the 24-dimensional Leech lattice or 32-dimensional Barnes–Wall ( $BW_{32}$ ) lattice, offer superior asymptotic packing density, their maximum likelihood decoding (MLD) complexity scales rapidly with the dimension, rendering them less suitable for real-time cryptographic deployments. In contrast, the 8-dimensional structure of  $E_8$  provides an optimal trade-off between coding gain and decoding complexity. It significantly improves the error-correction threshold over scalar nearest-neighbor decoding while maintaining constant-time, efficient decoding procedures. This design choice reduces the decryption failure rate (DFR) to a negligible level, allowing the system to operate with a lower modulus and a tighter noise budget without compromising efficiency or security.

## 2 General Algorithm Specification

This section defines the mathematical objects, byte encodings, arithmetic layer, public expansion routines, and algorithms that constitute *Viper*. The order is deliberately implementation-facing: background and notation come first, followed by encodings, arithmetic, PKE, KEM, parameters, and provenance of constants.

### 2.1 Notation

Let  $R = \mathbb{Z}[X]/(X^n + 1)$  and  $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ . In all parameter sets,  $n = 256$  and  $q = 2^{12}$  or  $q = 2^{13}$ . Elements of  $R_q$  are represented by coefficient vectors in  $\mathbb{Z}_q^n$ , the coefficient embedding is  $\tau(a_0 + \dots + a_{n-1}X^{n-1}) = (a_0, \dots, a_{n-1})$ . Vectors are column vectors,  $R_q^k$  denotes rank- $k$  modules and  $R_q^{m \times k}$  denotes  $m \times k$  matrices.

For a positive integer  $\eta$ , the centered binomial distribution  $\mathcal{B}_\eta$  is the distribution of  $\sum_{i=1}^\eta (a_i - b_i)$  for independent uniform bits  $a_i, b_i \leftarrow \{0, 1\}$ . For ring and module samples,  $\mathcal{B}_\eta$  is applied coefficient-wise, and  $\mathcal{B}_\eta^k$  denotes a vector in  $R_q^k$  with independent coefficients from  $\mathcal{B}_\eta$ . For  $a \in R_q$ , define  $\|a\|_\infty = \max_i |a_i|$ ; for  $\mathbf{v} \in R_q^k$ , define  $\|\mathbf{v}\|_\infty = \max_i \|\mathbf{v}[i]\|_\infty$ .

Let  $\Lambda \subset \mathbb{R}^n$  be a full-rank lattice. For  $\mathbf{y} \in \mathbb{R}^n$ , the nearest-neighbor quantizer [Zam14] is  $Q_\Lambda(\mathbf{y}) \triangleq \arg \min_{\lambda \in \Lambda} \|\mathbf{y} - \lambda\|$ . For a dither  $\mathbf{d} \in \mathbb{R}^n$ , define  $Q_{\Lambda, \mathbf{d}}(\mathbf{y}) \triangleq Q_\Lambda(\mathbf{y} + \mathbf{d})$ . The compensated output is  $Q_{\Lambda, \mathbf{d}}(\mathbf{y}) - \mathbf{d}$ , and the error is  $\mathbf{e} = Q_\Lambda(\mathbf{y} + \mathbf{d}) - (\mathbf{y} + \mathbf{d})$ . Transferred to  $R_q$  through  $\tau$ , for  $a \in R_q$  and  $d \in R_q$ :  $Q_{\Lambda, d}^\diamond(a) \triangleq \tau^{-1}(Q_{\Lambda, \tau(d)}(\tau(a))) \in R_q$ . The normal-form output is  $Q_{\Lambda, d}^\diamond(a) - d \in R_q$ , for vectors, the operator extends component-wise.

### 2.2 Message Encoding and Decoding

The PKE layer encrypts an FO plaintext  $m \in \{0, 1\}^{\ell_K}$ . The message codec maps this bit string to a polynomial over  $R_q$ , and maps a noisy polynomial back to a bit string during decryption. We write these two maps as  $\text{Encode} : \{0, 1\}^{\ell_K} \rightarrow R_q$  and  $\text{Decode} : R_q \rightarrow \{0, 1\}^{\ell_K}$ .

All submitted *Viper* profiles use an  $E_8$ -based block codec. Each profile fixes the number of active message blocks  $B_{\text{msg}}$ , an  $E_8$  scaling factor  $\alpha$ , and a block bit-width vector  $\beta = (\beta_0, \dots, \beta_7)$  satisfying  $\sum_{i=0}^7 \beta_i = \ell_K / B_{\text{msg}}$ . A field of width 0 is interpreted as the fixed integer 0, and positive-width fields are parsed in little-endian order. The remaining coefficients of the polynomial, if any, are set to zero by **Encode** and ignored by **Decode**.

The public block maps  $\text{Label}_{E_8}$  and  $\text{Delabel}_{E_8}$  are specified in Section 8.8. At this point, they are treated as fixed profile-dependent maps. The map  $\text{Label}_{E_8}$  embeds one parsed message block into an  $E_8$ -coded coefficient block in  $(\mathbb{Z}_q)^8$ , and  $\text{Delabel}_{E_8}$  recovers the parsed block after nearest-neighbor decoding in the scaled  $E_8$  lattice.

---

**Algorithm 1** Encode( $m$ )

---

**Require:** Message  $m = (m_0, \dots, m_{\ell_K-1}) \in \{0, 1\}^{\ell_K}$ .

**Ensure:** Encoded polynomial  $M \in R_q$ .

```
1:  $M \leftarrow 0 \in R_q$ .
2:  $\xi \leftarrow 0$ .
3: for  $j = 0$  to  $B_{\text{msg}} - 1$  do
4:   for  $i = 0$  to  $7$  do
5:      $a_i \leftarrow \sum_{h=0}^{\beta_i-1} m_{\xi+h} 2^h$ .
6:      $\xi \leftarrow \xi + \beta_i$ .
7:   end for
8:    $(\lambda_0, \dots, \lambda_7) \leftarrow \text{Label}_{E_8}(a_0, \dots, a_7)$ .
9:   for  $i = 0$  to  $7$  do
10:     $\tau(M)_{8j+i} \leftarrow \lambda_i$ .
11:   end for
12: end for
13: return  $M$ .
```

---

---

**Algorithm 2** Decode( $w$ )

---

**Require:** Polynomial  $w \in R_q$ .

**Ensure:** Message  $m' \in \{0, 1\}^{\ell_K}$ .

```
1:  $m' \leftarrow 0^{\ell_K}$ .
2:  $\xi \leftarrow 0$ .
3: for  $j = 0$  to  $B_{\text{msg}} - 1$  do
4:    $\mathbf{y} \leftarrow (\tau(w)_{8j}, \tau(w)_{8j+1}, \dots, \tau(w)_{8j+7}) \in (\mathbb{Z}_q)^8$ .
5:    $\boldsymbol{\lambda} \leftarrow \mathbf{Q}_{\alpha E_8}(\mathbf{y}) \bmod q\mathbb{Z}^8$ .
6:    $(a_0, \dots, a_7) \leftarrow \text{Delabel}_{E_8}(\boldsymbol{\lambda})$ .
7:   for  $i = 0$  to  $7$  do
8:     for  $h = 0$  to  $\beta_i - 1$  do
9:        $m'_{\xi+h} \leftarrow \lfloor a_i / 2^h \rfloor \bmod 2$ .
10:    end for
11:     $\xi \leftarrow \xi + \beta_i$ .
12:  end for
13: end for
14: return  $m'$ .
```

---

The nearest-neighbor step in Algorithm 2 is applied to centered coefficient representatives. Correct decoding of an active block occurs when the corresponding decryption-error block remains inside the Voronoi region of the scaled  $E_8$  decoding lattice. In the KEM, a decoding error is handled by the deterministic re-encryption check and implicit rejection.

### 2.3 Specification of Viper.PKE

The PKE layer is the deterministic-coin encryption primitive used by the FO transform. It exposes the split MLWQ-Z transcript through quantization labels and uses normal-form expansion before the corresponding coefficient-domain values participate in arithmetic.

The PKE is parameterized by  $(n, q, k, \eta_s, \eta_r, t_{\text{pk}}, t_u, t_v)$ . For  $x \in \{\text{pk}, \text{u}, \text{v}\}$ , define  $p_x = 2^{t_x}$ ,  $\Delta_x = q/p_x$ ,  $\Lambda_x = \Delta_x \mathbb{Z}^n$ . The public key uses  $p_{\text{pk}}$ , the ciphertext vector uses  $p_u$ , and the ciphertext scalar uses  $p_v$ . Let  $\mathcal{D}_x \subset R$  be the set of ring elements whose coefficients lie in  $\{0, \dots, \Delta_x - 1\}$ . The corresponding dither space is sampled coefficient-wise from  $\mathcal{D}_x$ .

For  $x \in \{\text{pk}, \text{u}, \text{v}\}$ , multiplication by  $\Delta_x$  gives a coefficient-wise bijection from  $R_{p_x}$  onto  $\tau^{-1}(\Lambda_x \cap \mathbb{Z}_q^n)$ . Whenever  $c \in \tau^{-1}(\Lambda_x \cap \mathbb{Z}_q^n)$ , the notation  $c/\Delta_x$  denotes the inverse of this bijection. It is

not division in  $R_q$ . Thus the algorithms may store a quantization label  $c/\Delta_x \in R_{p_x}$ , while the associated normal-form component is obtained by multiplying by  $\Delta_x$  and subtracting the public dither.

The public key is  $\mathbf{pk} = (\rho, \mathbf{b})$ , where  $\rho \in \{0, 1\}^{256}$  and  $\mathbf{b} \in R_{p_{pk}}^k$  is the public-key quantization label. The PKE secret key is  $\mathbf{sk}_{\text{PKE}} = \mathbf{s}$ . The ciphertext is  $\mathbf{ct} = (\mathbf{u}, v, \mu)$ , where  $\mathbf{u} \in R_{p_u}^k$ ,  $v \in R_{p_v}$ , and  $\mu \in \{0, 1\}^{256}$  is transmitted so that decapsulation can regenerate the ciphertext-side dithers.

The procedure  $\text{GenPublic}(\rho) \rightarrow (\mathbf{A}, \mathbf{d}_{\text{pk}})$  deterministically derives  $\mathbf{A} \in R_q^{k \times k}$  and  $\mathbf{d}_{\text{pk}} \in \mathcal{D}_{\text{pk}}^k$ . The procedure  $\text{GenDither}(\mu) \rightarrow (\mathbf{d}_u, d_v)$  deterministically derives  $\mathbf{d}_u \in \mathcal{D}_u^k$  and  $d_v \in \mathcal{D}_v$ . The distributions  $\mathcal{B}_{\eta_s}$  and  $\mathcal{B}_{\eta_r}$  are sampled coefficient-wise over  $R_q$ . The map  $\text{Encode} : \{0, 1\}^{\ell_K} \rightarrow R_q$  maps the profile message bits into the decoding lattice, and  $\text{Decode} : R_q \rightarrow \{0, 1\}^{\ell_K}$  applies the corresponding profile decoder.

All algebraic products are computed in  $R_q$  through the abstract routines `PolyMul`, `MatrixVectorMul`, and `InnerProd`. All quantization maps and normal-form expansions are applied coefficient-wise in the coefficient domain. The concrete multiplication backend is an implementation choice: the reference path is Toom–Cook/Karatsuba over  $R_q$ , while optimized implementations may use auxiliary NTT-friendly primes followed by CRT reconstruction without changing the scheme semantics.

---

### Algorithm 3 `Viper.PKE.KeyGen()`

---

**Require:** Parameter set  $(n, q, k, \eta_s, t_{\text{pk}})$ .

**Ensure:** Public key  $\mathbf{pk}$  and PKE secret key  $\mathbf{sk}_{\text{PKE}}$ .

- 1:  $\rho \leftarrow \mathcal{U}(\{0, 1\}^{256})$ .
  - 2:  $(\mathbf{A}, \mathbf{d}_{\text{pk}}) \leftarrow \text{GenPublic}(\rho)$ .
  - 3:  $\mathbf{s} \leftarrow \mathcal{B}_{\eta_s}^k$ .
  - 4:  $\mathbf{b} \leftarrow \mathbf{Q}_{\Lambda_{\text{pk}}, \mathbf{d}_{\text{pk}}}^\diamond(\mathbf{A}\mathbf{s})/\Delta_{\text{pk}}$ .
  - 5:  $\mathbf{pk} \leftarrow (\rho, \mathbf{b})$ .
  - 6:  $\mathbf{sk}_{\text{PKE}} \leftarrow \mathbf{s}$ .
  - 7: **return**  $(\mathbf{pk}, \mathbf{sk}_{\text{PKE}})$ .
- 

The seed  $\rho$  determines both the public matrix and the public-key dither. This keeps the public key compact and prevents the dither from becoming a separate transmitted object. The secret key is shown in coefficient representation. An implementation may store expanded local multiplication state for  $\mathbf{s}$  as an optimization, but this does not change the external secret-key format unless explicitly specified.

Encryption is written as  $\text{Viper.PKE.Enc}(\mathbf{pk}, m, \omega)$ . The coin string is parsed as  $\omega = (\sigma, \mu)$ . The seed  $\sigma$  derives the ephemeral secret. The seed  $\mu$  derives the ciphertext dithers and is transmitted as part of the ciphertext.

---

### Algorithm 4 `Viper.PKE.Enc(pk, m, ω)`

---

**Require:** Public key  $\mathbf{pk} = (\rho, \mathbf{b})$ , message  $m \in \{0, 1\}^{\ell_K}$ , and coins  $\omega \in \{0, 1\}^{512}$ .

**Ensure:** Ciphertext  $\mathbf{ct} = (\mathbf{u}, v, \mu)$ .

- 1:  $\omega \mapsto (\sigma, \mu) \in \{0, 1\}^{256} \times \{0, 1\}^{256}$ .
  - 2:  $(\mathbf{A}, \mathbf{d}_{\text{pk}}) \leftarrow \text{GenPublic}(\rho)$ .
  - 3:  $\widehat{\mathbf{b}} \leftarrow \Delta_{\text{pk}}\mathbf{b} - \mathbf{d}_{\text{pk}} \pmod{q}$ .
  - 4:  $\mathbf{r} \leftarrow \text{GenSecret}_{\eta_r}(\sigma)$ .
  - 5:  $(\mathbf{d}_u, d_v) \leftarrow \text{GenDither}(\mu)$ .
  - 6:  $\mathbf{u} \leftarrow \mathbf{Q}_{\Lambda_u, \mathbf{d}_u}^\diamond(\mathbf{A}^\top \mathbf{r})/\Delta_u$ .
  - 7:  $v \leftarrow \mathbf{Q}_{\Lambda_v, d_v}^\diamond(\widehat{\mathbf{b}}^\top \mathbf{r} + \text{Encode}(m))/\Delta_v$ .
  - 8:  $\mathbf{ct} \leftarrow (\mathbf{u}, v, \mu)$ .
  - 9: **return**  $\mathbf{ct}$ .
- 

The value  $\widehat{\mathbf{b}}$  is the normal form of the public-key split label. It satisfies  $\widehat{\mathbf{b}} = \mathbf{A}\mathbf{s} + \mathbf{e}_{\text{pk}}$  under the

input-independence theorem. The ciphertext vector is the split label of  $\mathbf{A}^\top \mathbf{r}$  under the ciphertext-vector dither. The scalar component is the split label of  $\hat{\mathbf{b}}^\top \mathbf{r} + \text{Encode}(m)$  under the scalar ciphertext dither.

---

**Algorithm 5**  $\text{Viper.PKE.Dec}(\text{sk}_{\text{PKE}}, \text{ct})$

---

**Require:** PKE secret key  $\text{sk}_{\text{PKE}} = \mathbf{s}$  and ciphertext  $\text{ct} = (\mathbf{u}, v, \mu)$ .

**Ensure:** Message  $m' \in \{0, 1\}^{\ell_K}$ .

- 1:  $(\mathbf{d}_u, d_v) \leftarrow \text{GenDither}(\mu)$ .
  - 2:  $\hat{\mathbf{u}} \leftarrow \Delta_u \mathbf{u} - \mathbf{d}_u \pmod{q}$ .
  - 3:  $\hat{v} \leftarrow \Delta_v v - d_v \pmod{q}$ .
  - 4:  $w \leftarrow \hat{v} - \mathbf{s}^\top \hat{\mathbf{u}}$ .
  - 5:  $m' \leftarrow \text{Decode}(w)$ .
  - 6: **return**  $m'$ .
- 

Correct decryption follows when each coefficient of  $w - \text{Encode}(m)$  remains inside the decoding interval. The error expression is  $w - \text{Encode}(m) = \mathbf{e}_{\text{pk}}^\top \mathbf{r} - \mathbf{s}^\top \mathbf{e}_u + e_v$ . This expression is the object used by the DFR computation.

## 2.4 Specification of Viper.KEM

The KEM layer is obtained by applying a Fujisaki–Okamoto transform with implicit rejection to the PKE layer. The resulting KEM uses deterministic encryption in encapsulation and in the decapsulation re-encryption check. Let  $\mathcal{H}$  and  $\mathcal{G}$  be cryptographic hash functions modeled as random oracles, with domain separation fixed by the implementation profile. All calls to  $\mathcal{H}$  are domain separated by their input role. We use the deterministic encryption procedure  $\text{Viper.PKE.Enc}(\text{pk}, m, \omega)$ , where  $\omega \mapsto (\sigma, \mu)$  is an explicit coin string. Let  $\mathcal{M}_{\text{FO}} = \{0, 1\}^{\ell_K}$ . Let  $\ell_K$  denote the shared-key output length selected by the KEM interface; the current parameter tables use  $\ell_K$  equal to the profile target. We write  $\mathcal{H}_{\ell_K}$  for the domain-separated hash output of length  $\ell_K$  bits.

The KEM secret key stores the PKE secret key, the public key, a hash of the public key, and a fallback seed. This follows the implicit-rejection style of FO transformations. The fallback seed prevents decapsulation failure from returning an externally visible failure symbol.

---

**Algorithm 6**  $\text{Viper.KEM.KeyGen}()$

---

**Ensure:** Public key  $\text{pk}$  and KEM secret key  $\text{sk}$ .

- 1:  $z \leftarrow \mathcal{U}(\{0, 1\}^{\ell_K})$ .
  - 2:  $(\text{pk}, \text{sk}_{\text{PKE}}) \leftarrow \text{Viper.PKE.KeyGen}()$ .
  - 3:  $h_{\text{pk}} \leftarrow \mathcal{H}(\text{pk})$ .
  - 4:  $\text{sk} \leftarrow (\text{sk}_{\text{PKE}}, \text{pk}, h_{\text{pk}}, z)$ .
  - 5: **return**  $(\text{pk}, \text{sk})$ .
- 

The public-key hash binds the encapsulation coins and the pre-key to the recipient public key. The final shared key is derived from the FO pre-key and the ciphertext hash.

---

**Algorithm 7** Viper.KEM.Encaps(pk)

---

**Require:** Public key pk.

**Ensure:** Ciphertext ct and shared key  $K \in \{0, 1\}^{\ell_K}$ .

- 1:  $h_{\text{pk}} \leftarrow \mathcal{H}(\text{pk})$ .
  - 2:  $m \leftarrow \mathcal{U}(\mathcal{M}_{\text{FO}})$ .
  - 3:  $(\hat{K}, \omega) \leftarrow \mathcal{G}(h_{\text{pk}}, m)$ .
  - 4:  $\text{ct} \leftarrow \text{Viper.PKE.Enc}(\text{pk}, m, \omega)$ .
  - 5:  $K \leftarrow \mathcal{H}_{\ell_K}(\hat{K}, \mathcal{H}(\text{ct}))$ .
  - 6: **return** (ct, K).
- 

The decapsulation algorithm always returns a key. It returns the real key if the deterministic re-encryption check succeeds. It returns a pseudorandom fallback key if the check fails. The comparison  $\text{ct} = \text{ct}'$  must be constant time. The selection between the two keys must be constant time.

---

**Algorithm 8** Viper.KEM.Decaps(sk, ct)

---

**Require:** Secret key  $\text{sk} = (\text{sk}_{\text{PKE}}, \text{pk}, h_{\text{pk}}, z)$  and ciphertext ct.

**Ensure:** Shared key  $K \in \{0, 1\}^{\ell_K}$ .

- 1:  $m' \leftarrow \text{Viper.PKE.Dec}(\text{sk}_{\text{PKE}}, \text{ct})$ .
  - 2:  $(\hat{K}', \omega') \leftarrow \mathcal{G}(h_{\text{pk}}, m')$ .
  - 3:  $\text{ct}' \leftarrow \text{Viper.PKE.Enc}(\text{pk}, m', \omega')$ .
  - 4: **if**  $\text{ct} = \text{ct}'$  **then**
  - 5:      $K \leftarrow \mathcal{H}_{\ell_K}(\hat{K}', \mathcal{H}(\text{ct}))$ .
  - 6: **else**
  - 7:      $K \leftarrow \mathcal{H}_{\ell_K}(z, \mathcal{H}(\text{ct}))$ .
  - 8: **end if**
  - 9: **return** K.
- 

The branch notation in Algorithm 8 is descriptive. A constant-time implementation must compute both candidate keys  $\mathcal{H}_{\ell_K}(\hat{K}', \mathcal{H}(\text{ct}))$  and  $\mathcal{H}_{\ell_K}(z, \mathcal{H}(\text{ct}))$ , and select the returned key with a mask. The equality test must compare the full ciphertext and must not terminate at the first mismatch.

### 3 Parameter Settings

This section outlines the definitive parameter configurations, structural byte sizes, and concrete security estimates for the Viper family. All specified profiles utilize a power-of-two modulus  $q$  and a fixed ring dimension  $n = 256$ . The module rank  $k$  scales the underlying module dimension, while the bit widths  $t_{\text{pk}}$ ,  $t_u$ , and  $t_v$  define the quantization levels for the public key, ciphertext vector, and ciphertext scalar components, respectively. For each component  $x \in \{\text{pk}, u, v\}$ , the compression modulus is denoted by  $p_x = 2^{t_x}$ , with the corresponding quantization step size given by  $\Delta_x = q/p_x$ .

The family consists of five core profiles targeting nominal security levels ranging from 128 to 512 bits, with the 192-bit and 384-bit rows serving as intermediate performance-security trade-offs.

#### 3.1 Selected Parameter Sets

Table 3 summarizes the foundational parameters for the Viper family. The secret distributions are centered binomial distributions  $\text{CBD}_{\eta_s}$  and  $\text{CBD}_{\eta_r}$ , instantiated with the profile-specific noise parameters.

Table 3: Parameter sets for the **Viper** family.

| Scheme    | Target $\lambda$ | $n$ | $q$      | $k$ | $(\eta_s, \eta_r)$ | $(t_{\text{pk}}, t_u, t_v)$ |
|-----------|------------------|-----|----------|-----|--------------------|-----------------------------|
| Viper-128 | 128              | 256 | $2^{12}$ | 2   | (2, 2)             | (9, 9, 4)                   |
| Viper-192 | 192              | 256 | $2^{12}$ | 3   | (3, 3)             | (10, 9, 6)                  |
| Viper-256 | 256              | 256 | $2^{12}$ | 4   | (3, 3)             | (10, 10, 5)                 |
| Viper-384 | 384              | 256 | $2^{13}$ | 7   | (1, 1)             | (11, 11, 5)                 |
| Viper-512 | 512              | 256 | $2^{13}$ | 9   | (1, 1)             | (11, 11, 8)                 |

### 3.2 Space Overheads and External Byte Lengths

The external representations of the protocol objects are structured as follows: the public key is  $\text{pk} = (\rho, \mathbf{b})$ , the ciphertext is  $\text{ct} = (\mathbf{u}, v, \mu)$ , and the KEM secret key is  $\text{sk} = (\text{sk}_{\text{PKE}}, \text{pk}, h_{\text{pk}}, z)$ . The public seeds  $\rho$  and  $\mu$  each occupy 32 bytes.

The concrete byte lengths are derived analytically from the coefficient bit-packing formulas:

$$|\text{pk}| = 32 + \frac{knt_{\text{pk}}}{8} \text{ bytes} \quad |\text{ct}| = 32 + \frac{knt_u + nt_v}{8} \text{ bytes}.$$

The PKE secret key  $\text{sk}_{\text{PKE}}$  is the long-term secret vector  $\mathbf{s}$ . In the submitted external format,  $\mathbf{s}$  is stored in expanded coefficient representation modulo  $q$ , rather than in a compressed CBD seed or sparse encoding. Thus one secret coefficient occupies  $\log_2 q$  bits, and

$$|\text{sk}_{\text{PKE}}| = \frac{kn \log_2 q}{8} \text{ bytes}.$$

The KEM secret key additionally stores the public key  $\text{pk}$ , the 32-byte public-key hash  $h_{\text{pk}}$ , and the fallback rejection string  $z$ . The fallback string has the same length as the shared secret. Consequently,

$$|\text{sk}| = |\text{sk}_{\text{PKE}}| + |\text{pk}| + 32 + |\text{ss}| = \frac{kn \log_2 q}{8} + |\text{pk}| + 32 + |\text{ss}|.$$

Table 4 records the exact external byte lengths derived under this layout.

Table 4: External byte lengths for the **Viper** parameter sets.

| Scheme    | $ \text{pk} $ | $ \text{ct} $ | $ \text{sk} $ | $ \text{ss} $ |
|-----------|---------------|---------------|---------------|---------------|
| Viper-128 | 608           | 736           | 1424          | 16            |
| Viper-192 | 992           | 1088          | 2200          | 24            |
| Viper-256 | 1312          | 1472          | 2912          | 32            |
| Viper-384 | 2496          | 2656          | 5488          | 48            |
| Viper-512 | 3200          | 3456          | 7040          | 64            |

### 3.3 Decryption Correctness and Security Summary

The compression widths  $(t_{\text{pk}}, t_u, t_v)$  are security-critical protocol constants. Changing these widths changes the induced quantization-error distributions, and therefore affects both the decryption-failure rate and the hardness of the associated lattice instances.

Table 5 reports the decryption-failure probability  $\log_2 \delta_{\text{dec}}$  together with concrete bit-security estimates. The Core-SVP columns follow the standard concrete-estimation methodology for LWE-style lattice attacks, using classical and quantum SVP cost models as in the LWE estimator methodology [APS15]. The MATZOV column reports the refined classical estimate under the improved dual-lattice attack model of the MATZOV report [MAT]. The scalar ciphertext component  $v$  is explicitly accounted for in the correctness and deterministic re-encryption analyses, but is not modeled as an independent module-recovery instance.

Table 5: Security estimates and decryption-failure rates for the **Viper** parameter sets. The **Core-SVP** columns report classical and quantum bit costs, respectively; the **MATZOV** column reports the refined classical estimate.

| Scheme    | Target $\lambda$ | Core-SVP (C/Q) | MATZOV (C) | $\log_2 \delta_{\text{dec}}$ |
|-----------|------------------|----------------|------------|------------------------------|
| Viper-128 | 128              | 118.6/108.7    | 141.9      | -177.91                      |
| Viper-192 | 192              | 177.5/162.1    | 199.4      | -172.22                      |
| Viper-256 | 256              | 246.2/225.3    | 266.0      | -272.99                      |
| Viper-384 | 384              | 393.7/361.1    | 410.2      | -351.83                      |
| Viper-512 | 512              | 549.5/521.0    | 564.6      | -421.06                      |

## 4 Performance Evaluation

This section evaluates the computational efficiency, memory footprint, and communication bandwidth of the **Viper** reference and optimized implementations across x86 and ARM Cortex-M4 architectures. For x86, the main results in this section report the formal **API\_PKC**-integrated submission implementations. In particular, the measured x86 entry points are the NGCC KEM API functions `kem_keygen`, `kem_enc`, and `kem_dec`, and the implementation path uses the official **API\_PKC** auxiliary functions and DRNG interface. Additional native-backend benchmark results, measured outside the formal **API\_PKC** wrapper, are reported separately in Appendix D.

### 4.1 Experimental Setup and Platforms

Benchmarking was conducted on a high-performance x86-64 desktop platform and on a Cortex-M4 microcontroller target using the `pqm4` benchmarking framework. The hardware environments, toolchains, timing sources, and measurement conventions are summarized in Table 6.

### 4.2 Implementation Variants

The software architecture provides platform-specific implementation variants tailored to portability, vectorized desktop performance, and embedded deployment.

- **Reference:** A portable C implementation targeting architectural flexibility. Polynomial multiplication is resolved using optimized Toom–Cook and Karatsuba block decomposition frameworks [MKV20], requiring no vector ISA support. In the formal **API\_PKC** submission, this implementation is compiled as ISO C99 and uses the official NGCC auxiliary functions and DRNG interface.
- **Optimized x86:** An optimized x86 implementation using AVX2 vector instructions. The formal **API\_PKC** optimized implementation is compiled through the submitted instance Makefiles and exposes only the NGCC KEM API functions. It uses the same external byte formats, the same official auxiliary-function interface, and the same KAT generator as the reference implementation.
- **Native x86 backends:** For additional implementation evidence, the native benchmarking path supports a vectorized Toom–Cook backend and an auxiliary double-prime NTT backend [CHK<sup>+</sup>21] over the coprime moduli 7681 and 10753, followed by CRT reconstruction. These native measurements are reported only in Appendix D; they are not the formal **API\_PKC** self-evaluation numbers.
- **Cortex-M4:** Embedded implementations are evaluated through the `pqm4` framework. The `m4fspeed` backend prioritizes cycle count, the `m4fstack` backend prioritizes reduced stack usage, and the `ref` backend provides a portable baseline.

Table 6: Hardware specifications and benchmarking environments.

| Component           | x86-64                                                                                                       | ARM Cortex-M4                                                 |
|---------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| CPU / MCU           | AMD Ryzen 7 9700X 8-Core Processor                                                                           | STM32L4R5ZIT6P, Arm Cortex-M4F                                |
| Board / Platform    | Desktop x86-64 platform                                                                                      | STM32 Nucleo-L4R5ZI-P, nucleo-l4r5zi                          |
| Memory              | N/A                                                                                                          | 2 MB Flash, 640 KB SRAM                                       |
| Clock frequency     | Cycle counts reported directly; no throughput conversion is used                                             | 20 MHz for CLOCK_BENCHMARK<br>120 MHz for CLOCK_FAST          |
| OS / Framework      | Ubuntu 22.04 series, Linux 6.8.0-124-generic                                                                 | pqm4 framework                                                |
| Toolchain           | GCC 11.4.0, GNU Make 4.3, CMake 3.22.1                                                                       | arm-none-eabi-gcc 10.3.1                                      |
| Reference flags     | -std=c99 -Wpedantic -Wall -Wextra -O2                                                                        | -O3 -g3<br>-opt speed                                         |
| Cortex-M4 flags     | N/A                                                                                                          | -mcpu=cortex-m4 -mthumb<br>-mfloat-abi=hard -mfpu=fpv4-sp-d16 |
| Optimized x86 flags | -O3 -march=x86-64 -mavx2<br>-mtune=native -flto<br>-fomit-frame-pointer<br>-std=c99 -Wpedantic -Wall -Wextra | N/A                                                           |
| Timing source       | Serialized x86 cycle counter in benchmark_kem_api.c                                                          | pqm4 speed benchmark using SysTick and hal_get_time()         |
| Statistics          | Median over 1000 measured executions after 100 warm-up executions                                            | Average over 10 executions                                    |
| Memory metric       | N/A                                                                                                          | Peak stack usage in bytes from pqm4 stack benchmark           |
| Code-size metric    | N/A                                                                                                          | .text, .data, and .bss<br>via arm-none-eabi-size              |

### 4.3 x86 Implementation Results

Table 7 reports the formal x86 API\_PKC self-evaluation results. The benchmark measures the public API functions `kem_keygen`, `kem_enc`, and `kem_dec`. The reported values are median cycle counts over 1000 measured executions after 100 warm-up executions. Both the reference and optimized implementations are evaluated through the same API\_PKC wrapper layer and use the official `drng.c/drng.h` and `auxfunc.c/auxfunc.h` integration path.

The optimized API\_PKC implementation improves over the reference implementation across all parameter sets. The measured speedups range from  $1.33\times$  to  $1.98\times$  for key generation, from  $1.35\times$  to  $2.08\times$  for encapsulation, and from  $1.46\times$  to  $2.20\times$  for decapsulation. The absolute cycle counts are higher than the native-backend numbers in Appendix D, because the formal API\_PKC evaluation includes the official auxiliary-function integration path and measures the submitted API wrapper directly.

### 4.4 Cortex-M4 Implementation Results

Table 8 reports Cortex-M4 KEM cycle counts for the `m4fspeed`, `m4fstack`, and `ref` backends. The reported values are averages over 10 executions and are given in kCycles. Table 9 reports the corresponding memory usage, hashing share, and linked binary size.

### 4.5 Comparison to Other Lattice-Based KEMs

To contextualize the practical efficiency and structural properties of Viper within the contemporary post-quantum cryptography landscape, we compare it with representative lattice-based KEMs. A

Table 7: Formal API\_PKC x86 KEM latency for Viper on AMD Ryzen 7 9700X, measured in clock cycles. Values are medians over 1000 measured runs after 100 warm-up runs.

| Scheme    | $\lambda$ | Implementation | KeyGen    | Encaps    | Decaps    |
|-----------|-----------|----------------|-----------|-----------|-----------|
| Viper-128 | 128       | Reference      | 157,317   | 224,195   | 269,034   |
|           | 128       | Optimized      | 118,558   | 166,436   | 184,296   |
| Viper-192 | 192       | Reference      | 294,114   | 391,011   | 453,711   |
|           | 192       | Optimized      | 202,915   | 269,414   | 286,324   |
| Viper-256 | 256       | Reference      | 477,269   | 598,107   | 676,765   |
|           | 256       | Optimized      | 303,994   | 378,472   | 394,051   |
| Viper-384 | 384       | Reference      | 1,433,330 | 1,934,919 | 2,094,895 |
|           | 384       | Optimized      | 811,473   | 929,461   | 950,739   |
| Viper-512 | 512       | Reference      | 2,611,684 | 2,881,668 | 3,043,354 |
|           | 512       | Optimized      | 1,320,662 | 1,517,117 | 1,544,097 |

Table 8: Cortex-M4 KEM speed results. Values are averages over 10 executions and are reported in kCycles.

| Scheme    | Implementation | KeyGen | Encaps | Decaps |
|-----------|----------------|--------|--------|--------|
| Viper-128 | m4fspeed       | 361    | 529    | 565    |
|           | m4fstack       | 382    | 531    | 568    |
|           | ref            | 1,074  | 1,675  | 2,102  |
| Viper-192 | m4fspeed       | 758    | 973    | 1,020  |
|           | m4fstack       | 815    | 976    | 1,023  |
|           | ref            | 2,345  | 3,209  | 3,849  |
| Viper-256 | m4fspeed       | 1,228  | 1,508  | 1,593  |
|           | m4fstack       | 1,341  | 1,510  | 1,596  |
|           | ref            | 4,068  | 5,187  | 6,034  |
| Viper-384 | m4fspeed       | 3,339  | 3,784  | 3,970  |
|           | m4fstack       | 3,732  | 3,787  | 3,974  |
|           | ref            | 12,121 | 14,032 | 15,489 |
| Viper-512 | m4fspeed       | 5,280  | 5,829  | 6,055  |
|           | m4fstack       | 5,951  | 5,833  | 6,061  |
|           | ref            | 19,826 | 22,267 | 24,119 |

native-interface comparison with representative lattice-based KEMs is reported in Appendix D. The main text reports the formal API\_PKC self-evaluation results in Table 7; the appendix uses the native benchmarking layer to match the comparison methodology used by prior schemes. The comparison shows that Viper achieves compact public keys and ciphertexts among module-structured primitives while maintaining competitive computational performance under the submitted API interface.

#### 4.5.1 Standard Unstructured LWE-based Schemes

Schemes based directly on the unstructured LWE problem, such as FrodoKEM [BCD<sup>+</sup>16] and SCloud<sup>+</sup> [WZZ<sup>+</sup>24], eschew ring or module symmetries, thereby relying on one of the most conservative and thoroughly scrutinized lattice assumptions in post-quantum cryptography. However, the absence of algebraic structure incurs a severe penalty in practical deployment. As illustrated in Table 14, unstructured LWE encryption typically yields public keys and ciphertexts that are an order of magnitude larger than those of structured variants; for instance, FrodoKEM-640 requires public keys nearing 10 kilobytes, compared to 608 bytes for Viper-128. Furthermore, their computational costs are substantially higher because operations involve large integer matrices rather than fast polynomial rings.

To mitigate these overheads within the unstructured paradigm, recent advances have explored

Table 9: Cortex-M4 KEM resource profile. Memory and binary-size components are reported in bytes; hashing share is reported in percent. Total denotes the linked binary size.

| Scheme    | Implementation | Memory |         |         | Hashing share |        |        | Binary size |       |      |        |
|-----------|----------------|--------|---------|---------|---------------|--------|--------|-------------|-------|------|--------|
|           |                | KeyGen | Encaps  | Decaps  | KeyGen        | Encaps | Decaps | .text       | .data | .bss | Total  |
| Viper-128 | m4fspeed       | 8,208  | 8,304   | 8,264   | 60.8%         | 66.3%  | 56.3%  | 32,355      | 0     | 0    | 32,355 |
|           | m4fstack       | 7,696  | 8,304   | 8,264   | 57.5%         | 66.0%  | 56.0%  | 31,403      | 0     | 0    | 31,403 |
|           | ref            | 11,460 | 14,732  | 15,100  | 20.5%         | 21.0%  | 15.2%  | 20,571      | 0     | 292  | 20,863 |
| Viper-192 | m4fspeed       | 10,312 | 10,192  | 10,152  | 60.6%         | 63.0%  | 54.8%  | 32,459      | 0     | 0    | 32,459 |
|           | m4fstack       | 8,776  | 10,192  | 10,152  | 56.4%         | 62.8%  | 54.6%  | 31,439      | 0     | 0    | 31,439 |
|           | ref            | 17,660 | 22,540  | 23,068  | 19.6%         | 19.1%  | 14.5%  | 21,503      | 0     | 292  | 21,795 |
| Viper-256 | m4fspeed       | 12,352 | 12,040  | 11,992  | 60.5%         | 61.0%  | 52.9%  | 32,739      | 0     | 0    | 32,739 |
|           | m4fstack       | 9,792  | 12,040  | 11,992  | 55.4%         | 60.9%  | 52.8%  | 32,067      | 0     | 0    | 32,067 |
|           | ref            | 25,020 | 30,980  | 31,404  | 18.3%         | 17.8%  | 14.0%  | 21,511      | 0     | 292  | 21,803 |
| Viper-384 | m4fspeed       | 18,536 | 17,768  | 17,704  | 60.1%         | 60.3%  | 54.1%  | 33,319      | 0     | 292  | 33,611 |
|           | m4fstack       | 12,904 | 17,876  | 17,704  | 53.8%         | 60.3%  | 54.1%  | 31,931      | 0     | 292  | 32,223 |
|           | ref            | 59,444 | 68,676  | 69,020  | 16.6%         | 16.3%  | 13.9%  | 22,363      | 0     | 292  | 22,655 |
| Viper-512 | m4fspeed       | 22,632 | 21,604  | 21,516  | 60.0%         | 60.0%  | 54.7%  | 33,379      | 0     | 292  | 33,671 |
|           | m4fstack       | 14,952 | 21,496  | 21,408  | 53.3%         | 60.0%  | 54.7%  | 31,995      | 0     | 292  | 32,287 |
|           | ref            | 91,316 | 102,708 | 103,228 | 16.0%         | 15.8%  | 13.8%  | 22,323      | 0     | 292  | 22,615 |

the Unstructured Learning with Quantization (ULWQ) framework, exemplified by the Frost family. By incorporating quantization natively, Frost achieves a more compact bandwidth profile than FrodoKEM while preserving standard lattice guarantees. Parallel to this line of work, **Viper** takes a different structural path by leveraging a module setting to compress public artifacts. Crucially, through the MLWQ framework, **Viper** retains a tight reduction to Module LWE (MLWE), combining the structural flexibility of modules with the robust correctness profiles driven by public dithered quantization.

#### 4.5.2 MLWE- and MLWR-based Schemes

The most widely deployed structured lattice KEMs are built upon the Module Learning with Errors (MLWE) and Module Learning with Rounding (MLWR) problems, with ML-KEM (Kyber) [BDK<sup>+</sup>18] and Saber [DKRV18] serving as their respective archetypes. MLWE-based schemes rely on explicitly sampled error distributions, an approach that facilitates tight security reductions and has become the dominant design paradigm in modern PQC standards. However, communication compression in these schemes is typically treated as an auxiliary post-processing step, implemented via rounding operations that are separate from the underlying hardness assumption.

Conversely, MLWR-based constructions replace explicit error sampling with deterministic rounding, eliminating discrete samplers and yielding compact communication artifacts, as seen in the **Sable/Saber** series. Nevertheless, this deterministic rounding error inherently depends on the secret-dependent quantity being rounded, which prevents direct reductions to MLWE and introduces heuristic gaps in their formal security proofs.

**Viper** occupies an intermediate position between these two paradigms. Its underlying MLWQ assumption uses public dithered quantization to generate quantization noise that is statistically independent of the hidden secret. Consequently, **Viper** retains the communication compactness and sampling-light structure of MLWR-style primitives, while preserving a non-heuristic security reduction to MLWE. From an engineering perspective, **Viper** remains compatible with the optimized polynomial pipelines developed for module-lattice schemes, combining competitive polynomial arithmetic with compact public artifacts.

### 4.5.3 RLWE-, RLWR- and NTRU-based Schemes

Ideal or ring-based constructions, including those utilizing Ring LWE such as LAC [LLZ<sup>+</sup>18] and the NTRU assumption such as NEV [ZFY23], represent another important direction in lattice-based KEM design. These schemes operate directly over a monolithic polynomial ring, collapsing matrix dimensions to unity and reducing representation overhead because public matrices degenerate into single ring elements.

However, this monolithic algebraic structure imposes a strict rigidity: both the security margin and the implementation efficiency are tied to the degree of the underlying ring. Scaling security levels in a ring-based KEM typically requires modifying the ring dimension itself, which in turn requires retuning polynomial arithmetic and implementation pipelines.

In contrast, module-based constructions such as **Viper** offer greater algebraic flexibility. They scale across security levels primarily by varying the module rank  $k$ , while preserving the same underlying ring arithmetic and hardware primitives. Furthermore, while NTRU-based schemes benefit from efficient and compact encapsulation, their algebraic properties diverge from the MLWE lineage and require distinct cryptanalytic validation. By adopting the module-lattice approach through the lens of the MLWQ framework, **Viper** preserves the modular design philosophy of contemporary standards while elevating quantization to a native, mathematically explicit component of the cryptographic construction.

## 5 Known Answer Tests

The official NGCC known-answer tests for **Viper** are included under `API_PKC/Test_Vectors/` in the submission package. The submitted `API_PKC` package contains one Reference instance and one Optimized instance for each submitted **Viper** parameter set.

The relevant package layout is as follows:

```
API_PKC/  
  README  
  Implementations/  
    Reference_Implementation/  
      AlgorithmInstance/  
        KAT_KEM.c  
        drng.c  
        drng.h  
        auxfunc.c  
        auxfunc.h  
      MAMBA-Viper-128/  
      MAMBA-Viper-192/  
      MAMBA-Viper-256/  
      MAMBA-Viper-384/  
      MAMBA-Viper-512/  
    Optimized_Implementation/  
      MAMBA-Viper-128/  
      MAMBA-Viper-192/  
      MAMBA-Viper-256/  
      MAMBA-Viper-384/  
      MAMBA-Viper-512/  
  Test_Vectors/  
    KAT_KEM_MAMBA-Viper-128.txt  
    KAT_KEM_MAMBA-Viper-192.txt  
    KAT_KEM_MAMBA-Viper-256.txt  
    KAT_KEM_MAMBA-Viper-384.txt  
    KAT_KEM_MAMBA-Viper-512.txt  
  Internal_Tests/  
    NGCC_API/  
      api_check.c
```

Each submitted instance directory contains the NGCC API\_PKC KEM wrapper, the official KAT generator, and the auxiliary support files required by the interface. In particular, the files `KAT_KEM.c`, `drng.c`, `drng.h`, `auxfunc.c`, and `auxfunc.h` are copied into each submitted instance directory. These files are byte-for-byte identical to the corresponding files in `API_PKC/Implementations/Reference_Implementation/AlgorithmInstance/`. The official auxiliary-function interface provides `sm3hash`, `pseudohash`, and `pseudoXOF`, and the official DRNG interface is used through `drng.c/drng.h`.

For each parameter set, running `make clean`, `make`, `make kat` inside a Reference or Optimized instance directory builds the API instance and executes the official KAT generator. The target `make kat` writes an instance-local `output/KAT_KEM_MAMBA-Viper-XXX.txt` file and copies the formal KAT output to `API_PKC/Test_Vectors/`. The Reference output is used as the formal KAT value, and the Optimized output is required to be byte-identical to the corresponding Reference output.

Table 10: Official NGCC API\_PKC KAT files for the submitted Viper instances.

| Instance        | pk   | sk   | ct   | ss | KAT file                                         |
|-----------------|------|------|------|----|--------------------------------------------------|
| MAMBA-Viper-128 | 608  | 1424 | 736  | 16 | API_PKC/Test_Vectors/KAT_KEM_MAMBA-Viper-128.txt |
| MAMBA-Viper-192 | 992  | 2200 | 1088 | 24 | API_PKC/Test_Vectors/KAT_KEM_MAMBA-Viper-192.txt |
| MAMBA-Viper-256 | 1312 | 2912 | 1472 | 32 | API_PKC/Test_Vectors/KAT_KEM_MAMBA-Viper-256.txt |
| MAMBA-Viper-384 | 2496 | 5488 | 2656 | 48 | API_PKC/Test_Vectors/KAT_KEM_MAMBA-Viper-384.txt |
| MAMBA-Viper-512 | 3200 | 7040 | 3456 | 64 | API_PKC/Test_Vectors/KAT_KEM_MAMBA-Viper-512.txt |

Each KAT file consists of a sequence of deterministic test records. A record contains the deterministic test seed, the generated public key, secret key, ciphertext, and shared secret, together with their byte lengths. The fields are written as `Count`, `Seed_Len`, `Seed`, `PK_Len`, `PK`, `SK_Len`, `SK`, `CT_Len`, `CT`, `SS_Len`, and `SS`. The byte lengths in the KAT records match the external API lengths reported in Table 10.

The directory `API_PKC/Internal_Tests/NGCC_API/` contains the internal API conformance test used to check the submitted `kem_*` interface. Consequently, the submitted KAT files are produced through the same NGCC API\_PKC interface used for conformance testing. The Reference and Optimized implementations expose the same public API, produce byte-identical KAT outputs for the same deterministic test seeds, and use the same external public-key, secret-key, ciphertext, and shared-secret lengths.

## 6 Security Strength

This section analyzes the security of Viper. We state the IND-CPA security of `Viper.PKE` under the normal-form MLWE assumptions induced by MLWQ, and the IND-CCA security of `Viper.KEM` through the Fujisaki–Okamoto transform in the ROM and QROM. We then report concrete lattice-security estimates for the submitted parameter sets and discuss relevant attack surfaces.

### 6.1 Security Definitions and Claims

**Public Key Encryption.** A public key encryption scheme PKE consists of a tuple of polynomial-time algorithms (`KeyGen`, `Enc`, `Dec`). The key-generation algorithm `KeyGen`( $1^\lambda$ ) outputs a public key `pk` and a secret key `sk`. The encryption algorithm `Enc`(`pk`,  $m$ ) takes  $m \in \mathcal{M}$  and outputs a ciphertext `ct`. The decryption algorithm `Dec`(`sk`, `ct`) outputs a message  $m'$  or a failure symbol  $\perp$ .

**$\delta$ -Correctness.** A PKE scheme is  $\delta$ -correct if for all  $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$  and all  $m \in \mathcal{M}$ ,

$$\Pr[\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m)) \neq m] \leq \delta.$$

The probability is taken over the randomness of `Enc`. For `Viper.PKE`, the target regime is a cryptographically negligible decryption-failure probability.

**IND-CPA Security.** For a PKE adversary  $\mathcal{A}$ , define

$$\mathbf{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr \left[ \mathcal{A}^{\text{Enc}(\text{pk}, \cdot)}(\text{pk}, \text{ct}^*) = b \right] - \frac{1}{2} \right|,$$

where  $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}$ , the adversary chooses  $m_0, m_1$ , the challenger samples  $b \leftarrow \mathcal{U}(\{0, 1\})$ , and returns  $\text{ct}^* \leftarrow \text{Enc}(\text{pk}, m_b)$ .

**Key Encapsulation Mechanism.** A KEM scheme  $\text{KEM}$  consists of a tuple of algorithms  $(\text{KeyGen}, \text{Encaps}, \text{Decaps})$ . The key-generation algorithm  $\text{KeyGen}(1^\lambda)$  outputs a public key  $\text{pk}$  and a secret key  $\text{sk}$ . The encapsulation algorithm  $\text{Encaps}(\text{pk})$  outputs a ciphertext  $\text{ct}$  and a shared key  $K \in \mathcal{K}$ . The decapsulation algorithm  $\text{Decaps}(\text{sk}, \text{ct})$  outputs a shared key  $K' \in \mathcal{K}$ . In implicit-rejection KEMs such as  $\text{Viper.KEM}$ , malformed ciphertexts are mapped to a pseudorandom fallback key rather than to an externally visible failure symbol.

**IND-CCA Security.** For a KEM adversary  $\mathcal{A}$  with access to a decapsulation oracle  $\mathcal{O}_{\text{dec}}$ , define

$$\mathbf{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) = \left| \Pr \left[ \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda), \quad b \leftarrow \mathcal{U}(\{0, 1\}), \\ (\text{ct}^*, K_0) \leftarrow \text{Encaps}(\text{pk}), \quad K_1 \leftarrow \mathcal{U}(\mathcal{K}), \quad : b' = b \\ b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{dec}}}(\text{pk}, \text{ct}^*, K_b) \end{array} \right] - \frac{1}{2} \right|.$$

The adversary is not allowed to query  $\mathcal{O}_{\text{dec}}$  on  $\text{ct}^*$ .

This section records the formal security claims made by the  $\text{Viper}$  submission. These claims are the basis for the security analysis in the remainder of this document.

**Claim 6.1** (IND-CCA Security in ROM). *The  $\text{Viper.KEM}$  construction is IND-CCA secure in the classical random-oracle model under the hardness of the MLWQ problem.*

**Claim 6.2** (IND-CCA Security in QROM). *The  $\text{Viper.KEM}$  construction is IND-CCA secure in the quantum random-oracle model under the hardness of the MLWQ problem.*

**Claim 6.3** (Concrete security estimates). *The five submitted  $\text{Viper}$  profiles target nominal security levels 128, 192, 256, 384, and 512. Table 11 reports the corresponding concrete lattice estimates under three models: the MATZOV classical model, the Core-SVP classical baseline, and the Core-SVP quantum sieve model. The MATZOV classical estimates exceed the nominal targets for all five profiles; the Core-SVP estimates are reported separately as conservative baseline and quantum comparison values.*

## 6.2 Correctness of the PKE Scheme

We analyze correctness through the compensated normal form of the split MLWQ samples. For  $x \in \{\text{pk}, \text{u}, \text{v}\}$ , let  $p_x = 2^{t_x}$ ,  $\Delta_x = q/p_x$ , and  $\chi_x = \chi_{q, p_x}$ . The reconstructed public-key and ciphertext components satisfy  $\hat{\mathbf{b}} = \Delta_{\text{pk}} \mathbf{b} - \mathbf{d}_{\text{pk}} = \mathbf{A} \mathbf{s} + \mathbf{e}_{\text{pk}}$ ,  $\hat{\mathbf{u}} = \Delta_{\text{u}} \mathbf{u} - \mathbf{d}_{\text{u}} = \mathbf{A}^T \mathbf{r} + \mathbf{e}_{\text{u}}$ , and  $\hat{\mathbf{v}} = \Delta_{\text{v}} \mathbf{v} - \mathbf{d}_{\text{v}} = \hat{\mathbf{b}}^T \mathbf{r} + \text{Encode}(m) + e_{\text{v}}$ . The entries of  $\mathbf{e}_{\text{pk}}$  and  $\mathbf{e}_{\text{u}}$  are polynomials whose coefficients follow  $\chi_{\text{pk}}$  and  $\chi_{\text{u}}$ , respectively, while the coefficients of  $e_{\text{v}}$  follow  $\chi_{\text{v}}$ . For  $q = 2^\ell$ ,  $p = 2^t$ , and  $\Delta = q/p$ , the scalar quantization-error distribution is  $\chi_{q, p} = \mathcal{U}(\{-\Delta/2 + 1, \dots, \Delta/2\})$ .

Decryption computes  $w = \hat{\mathbf{v}} - \mathbf{s}^T \hat{\mathbf{u}}$ . Substituting the normal forms gives

$$w = \text{Encode}(m) + e_{\text{dec}}, \quad e_{\text{dec}} = \mathbf{e}_{\text{pk}}^T \mathbf{r} - \mathbf{s}^T \mathbf{e}_{\text{u}} + e_{\text{v}}.$$

Here  $e_{\text{dec}} \in R_q$ ,  $\mathbf{s} \leftarrow \mathcal{B}_{\eta_s}^k$ ,  $\mathbf{r} \leftarrow \mathcal{B}_{\eta_r}^k$ , and the coefficient distributions of  $\mathbf{e}_{\text{pk}}$ ,  $\mathbf{e}_{\text{u}}$ , and  $e_{\text{v}}$  are  $\chi_{\text{pk}}$ ,  $\chi_{\text{u}}$ , and  $\chi_{\text{v}}$ , respectively. Correctness is determined by whether the centered coefficient vector  $\tau(e_{\text{dec}}) \in \mathbb{Z}^n$  lies inside the decoding region of the  $E_8$  message codec.

We use a closed-form independent-coordinate model to express the main correctness condition. The distribution  $\chi_{q, p}$  has mean  $\mu_{q, p} = 1/2$ , variance  $\sigma_{q, p}^2 = (\Delta^2 - 1)/12$ , and second moment  $\nu_{q, p} = \mathbb{E}[\chi_{q, p}^2] = (\Delta^2 + 2)/12$ . Write  $\mu_x = \mu_{q, p_x}$ ,  $\sigma_x^2 = \sigma_{q, p_x}^2$ , and  $\nu_x = \nu_{q, p_x}$  for  $x \in \{\text{pk}, \text{u}, \text{v}\}$ .

Since  $\mathcal{B}_{\eta_s}$  and  $\mathcal{B}_{\eta_r}$  have mean 0 and variances  $\eta_s/2$  and  $\eta_r/2$ , a coefficient of  $e_{\text{dec}}$  has mean  $\mu_E = \mathbb{E}[e_{\text{dec},i}] = \mu_v = 1/2$ , and the scalar noise variance is

$$\sigma_{\text{tot}}^2 = kn \nu_{\text{pk}} \frac{\eta_r}{2} + kn \frac{\eta_s}{2} \nu_u + \frac{\Delta_v^2 - 1}{12}.$$

In this model, the coordinates of an  $E_8$  error block are treated as independent random variables  $E_i \sim N(\mu_E, \sigma_{\text{tot}}^2)$ .

For rate  $R$ , each  $E_8$  block encodes  $8R$  message bits and uses the fine decoding lattice  $\alpha E_8$ , where  $\alpha = q/2^R$ . The 128-, 192-, and 256-bit parameter sets use  $R = 1$ , while the 384- and 512-bit parameter sets use  $R = 2$ . If  $\ell_{\text{msg}}$  is the FO message length in bits, the number of active  $E_8$  blocks is  $B_{E_8} = \ell_{\text{msg}}/(8R)$ .

Let  $Q(x) = \frac{1}{\sqrt{2\pi}} \int_x^\infty e^{-t^2/2} dt$ . The  $E_8$  Voronoi-relevant roots have squared norm 2 and consist of the pair roots  $(\pm 1, \pm 1, 0, \dots, 0)$  and the half roots  $(\pm 1/2, \dots, \pm 1/2)$  with an even number of minus signs. For the pair roots, the three distinct projection means are  $2\mu_E$ , 0, and  $-2\mu_E$ , and all have variance  $2\sigma_{\text{tot}}^2$ . Thus

$$\delta_{\text{pair}}^G = \binom{8}{2} \left[ Q\left(\frac{\alpha - 2\mu_E}{\sqrt{2}\sigma_{\text{tot}}}\right) + 2Q\left(\frac{\alpha}{\sqrt{2}\sigma_{\text{tot}}}\right) + Q\left(\frac{\alpha + 2\mu_E}{\sqrt{2}\sigma_{\text{tot}}}\right) \right].$$

For a half root with  $m \in \{0, 2, 4, 6, 8\}$  minus signs, the projection has mean  $(4 - m)\mu_E$  and variance  $2\sigma_{\text{tot}}^2$ , giving

$$\delta_{\text{half}}^G = \sum_{m \in \{0, 2, 4, 6, 8\}} \binom{8}{m} Q\left(\frac{\alpha - (4 - m)\mu_E}{\sqrt{2}\sigma_{\text{tot}}}\right).$$

The resulting closed-form correctness estimate is

$$\delta_{\text{dec}}^G \leq B_{E_8} (\delta_{\text{pair}}^G + \delta_{\text{half}}^G).$$

While the Gaussian heuristic provides a clear algebraic dependence of the DFR on the system parameters, it loses precision in the deep tails of the distribution. Therefore, to ensure strict bounds, the numerical DFR values reported in Table 5 are evaluated using an exact discrete marginal-product estimator, which we detail in Appendix B.

### 6.3 Theoretical Security

We first state the IND-CPA security of the underlying PKE. The statement accounts for the concrete seeded implementation, where domain-separated XOF calls expand seeds into the public matrix, dithers, and deterministic encryption randomness. The detailed proofs of the security statements in this subsection are deferred to Appendix C. Let  $\chi_{\text{ct}} = (\chi_{q, p_u}^k, \chi_{q, p_v})$  denote the mixed ciphertext-error distribution for the  $k$  ciphertext-vector components and the scalar  $v$ -component.

**Theorem 6.1.** *For every PPT IND-CPA adversary  $\mathcal{A}$  against the seeded implementation of Viper.PKE, there exist PPT adversaries  $\mathcal{C}_{\text{XOF}}$ ,  $\mathcal{B}_{\text{pk}}$ , and  $\mathcal{B}_{\text{ct}}$  such that*

$$\begin{aligned} \text{Adv}_{\text{Viper.PKE}}^{\text{IND-CPA}}(\mathcal{A}) &\leq \text{Adv}_{\text{XOF}}(\mathcal{C}_{\text{XOF}}) + \text{Adv}_{q, k, k, \mathcal{B}_{\eta_s}, \chi_{q, p_{\text{pk}}}}^{\text{MLWE}}(\mathcal{B}_{\text{pk}}) \\ &\quad + \text{Adv}_{q, k, k+1, \mathcal{B}_{\eta_r}, \chi_{\text{ct}}}^{\text{MLWE}}(\mathcal{B}_{\text{ct}}). \end{aligned}$$

Here  $\text{Adv}_{\text{XOF}}$  denotes the distinguishing advantage for replacing the concrete domain-separated XOF expansions by ideal sampling of  $(\mathbf{A}, \mathbf{d}_{\text{pk}})$ ,  $(\mathbf{d}_u, \mathbf{d}_v)$ , and  $\mathbf{r}$  with the prescribed distributions.

We now lift the PKE security to KEM security through the Fujisaki–Okamoto transform with implicit rejection. Let  $\delta_{\text{dec}}$  denote the whole-ciphertext decryption-failure probability, and let  $q_{\text{RO}} \triangleq q_{\mathcal{G}} + q_{\mathcal{H}}$ .

Table 11: Consolidated concrete security estimates for the Viper parameter sets.

| Scheme    | Target | Core-SVP (C) | Core-SVP (Q) | MATZOV (C) |
|-----------|--------|--------------|--------------|------------|
| Viper-128 | 128    | 118.6        | 108.7        | 141.9      |
| Viper-192 | 192    | 177.5        | 162.1        | 199.4      |
| Viper-256 | 256    | 246.2        | 225.3        | 266.0      |
| Viper-384 | 384    | 393.7        | 361.1        | 410.2      |
| Viper-512 | 512    | 549.5        | 521.0        | 564.6      |

**Theorem 6.2.** *Assume that  $\mathcal{G}$  and  $\mathcal{H}$  are random oracles. For every classical adversary  $\mathcal{A}$  against Viper.KEM making at most  $q_{\mathcal{G}}$  queries to  $\mathcal{G}$ ,  $q_{\mathcal{H}}$  queries to  $\mathcal{H}$ , and  $q_D$  decapsulation queries, there exists an IND-CPA adversary  $\mathcal{B}$  against Viper.PKE such that*

$$\text{Adv}_{\text{Viper.KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq 3\text{Adv}_{\text{Viper.PKE}}^{\text{IND-CPA}}(\mathcal{B}) + (2q_{\text{RO}} + q_D)\delta_{\text{dec}} + \frac{3q_{\text{RO}}}{|\mathcal{M}_{\text{FO}}|}.$$

**Theorem 6.3.** *Assume that  $\mathcal{G}$  and  $\mathcal{H}$  are quantum-accessible random oracles, while decapsulation queries remain classical. For every quantum adversary  $\mathcal{A}$  against Viper.KEM making at most  $q_{\mathcal{G}}$  queries to  $\mathcal{G}$ ,  $q_{\mathcal{H}}$  queries to  $\mathcal{H}$ , and  $q_D$  decapsulation queries, there exists an IND-CPA adversary  $\mathcal{B}$  against Viper.PKE such that*

$$\text{Adv}_{\text{Viper.KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq \sqrt{\frac{2q_{\mathcal{H}}}{|\mathcal{M}_{\text{FO}}|}} + 4q_{\mathcal{G}}\sqrt{\delta_{\text{dec}}} + 2q_{\text{RO}}\sqrt{\text{Adv}_{\text{Viper.PKE}}^{\text{IND-CPA}}(\mathcal{B}) + \frac{1}{|\mathcal{M}_{\text{FO}}|}}.$$

The ROM bound follows the standard FO analysis with implicit rejection. The QROM bound contains the usual square-root degradation in both the decryption-failure term and the underlying PKE advantage.

## 6.4 Concrete Security

We estimate the concrete hardness of the normal-form MLWE instances using the `lattice-estimator` [ACD<sup>+</sup>18]. For each route, the module instance over  $R_q$  is expanded to a scalar LWE instance over  $\mathbb{Z}_q$  with secret dimension  $N = nk$  and  $M = nk$  equations. The public-key route uses secret distribution  $\mathcal{B}_{\eta_s}$  and error distribution  $\chi_{q,p_k}$ . The ciphertext-vector route uses secret distribution  $\mathcal{B}_{\eta_r}$  and error distribution  $\chi_{q,p_u}$ . For each parameter set, we take the weaker of the public-key and ciphertext-vector routes.

Table 11 consolidates the resulting estimates. The BDD, dual, dual-hybrid, and BKW columns report representative attack costs under the classical Core-SVP baseline [ADPS16]. The column C-SVP C gives the weakest classical Core-SVP estimate among these routes. The MATZOV column reports the refined classical estimate under the MATZOV model [GJ21, WSM<sup>+</sup>25]. The column C-SVP Q reports the corresponding quantum Core-SVP estimate under the quantum sieve model  $2^{0.265\beta}$ .

## 6.5 Side-Channel Security Considerations

Side-channel and implementation attacks are outside the lattice-estimator model. The reference design uses power-of-two arithmetic, fixed-width operations, and deterministic re-encryption in decapsulation, which are compatible with constant-time implementation. As usual, masked implementations and leakage testing are separate implementation-security tasks and do not change the MLWQ estimates reported above.

### 6.5.1 Timing Attacks

Timing attacks exploit execution-time variations that depend on secret data. A primary design goal of Viper is to avoid secret-dependent control flow and variable-time arithmetic. The scheme

operates entirely over power-of-two moduli and uses deterministic quantization functions. Consequently, no rejection sampling, modular inversion, or variable-time reduction modulo a prime is required. All quantization operations reduce to additions, shifts, masking, and fixed-latency arithmetic instructions. Furthermore, the MLWQ construction replaces explicit error sampling with deterministic dithered quantization. As a result, implementations avoid many timing leakages associated with discrete Gaussian samplers or complex centered-binomial generation procedures.

For these reasons, constant-time implementations of *Viper* can be realized using standard constant-time polynomial arithmetic and fixed-time compression routines.

### 6.5.2 Differential Power and Electromagnetic Analysis

Differential Power Analysis (DPA) and Differential Electromagnetic Analysis (DEMA) exploit statistical correlations between side-channel measurements and secret-dependent intermediate values. As with all lattice-based KEMs, the polynomial multiplications involving the long-term secret key constitute the primary attack surface. Standard masking techniques can therefore be applied to the secret polynomials and all intermediate arithmetic values. Compared with traditional MLWE schemes, *Viper* benefits from the absence of explicit error-generation procedures. In particular, no discrete Gaussian sampler must be protected, and no rejection-based sampling loop introduces additional leakage points. Moreover, decapsulation avoids additional secret-dependent recovery mechanisms; message recovery follows a fixed lattice decoding procedure with public parameters and a fixed execution structure. The dither values used in *Viper* are public and independent of the secret key. Therefore, the dither generation process does not introduce any additional secret-dependent leakage channels.

Overall, the masking complexity of *Viper* is comparable to that of Saber while eliminating several components that typically dominate the implementation cost of protected MLWE systems.

### 6.5.3 Single-Trace Attacks

Masking alone does not automatically provide resistance against single-trace side-channel attacks. Horizontal attacks can exploit multiple occurrences of secret-related operations within a single execution trace. The main targets in *Viper* are the polynomial multiplications involving the secret vectors during encapsulation and decapsulation. As in other module-lattice schemes, randomized shuffling, operation reordering, and dummy computations can be applied to reduce exploitable leakage. Unlike NTT-based constructions, *Viper* can be implemented using schoolbook, Karatsuba, or Toom–Cook multiplication techniques. These arithmetic structures offer natural opportunities for randomized execution ordering and fine-grained shuffling. Since the quantization and dequantization procedures consist only of simple arithmetic operations, their protection incurs negligible additional overhead compared with the protection of polynomial multiplication itself.

We therefore expect higher-order protected implementations of *Viper* to scale favorably and to achieve lower implementation complexity than schemes requiring protected Gaussian sampling, rejection sampling, or reconciliation logic.

## 7 Advantages and Limitations

Compared with existing MLWE- and MLWR-based KEMs, *Viper* offers several distinctive advantages while also introducing new design trade-offs. The following discussion summarizes the principal strengths and limitations of the current design.

### 7.1 Advantages

#### 1. Security transparency through input-independent noise.

The defining feature of *Viper* is the use of public-dithered quantization. The resulting quantization noise is statistically independent of the underlying lattice point and therefore behaves

similarly to the explicit error term in MLWE. This property enables security reductions that closely resemble those used in MLWE-based cryptography while preserving the communication benefits of quantization. From a standardization perspective, such transparency simplifies both cryptanalysis and parameter justification.

## 2. **Native communication compression.**

Unlike MLWE-based schemes, which first generate an error term and then compress the resulting ciphertext, *Viper* directly incorporates quantization into the encryption process. Compression is therefore not an auxiliary optimization but an integral part of the underlying hardness assumption. This approach allows ciphertext and public-key sizes to remain compact without introducing additional reconciliation mechanisms or heuristic noise-management techniques.

## 3. **Exact decryption-failure analysis.**

The induced error distribution in *Viper* is explicitly known and bounded. Consequently, decryption-failure probabilities can be computed directly from the underlying distributions without relying on Gaussian approximations or asymptotic estimates. This improves reproducibility and makes independent verification of correctness claims substantially easier.

## 4. **Implementation simplicity.**

The scheme avoids several components that traditionally dominate the implementation complexity of lattice-based cryptography. In particular, no rejection sampling or discrete Gaussian sampling is required. Most arithmetic operations reduce to additions, bit shifts, masking operations, and polynomial multiplications. As a result, both software implementations and hardware realizations remain comparatively simple.

## 5. **Side-channel friendly architecture.**

Many practical side-channel vulnerabilities arise from components such as Gaussian samplers, rejection loops, and reconciliation procedures. Because *Viper* avoids these mechanisms, the number of side-channel-sensitive building blocks is significantly reduced. Protected implementations can focus primarily on secret-dependent polynomial arithmetic, which simplifies masking and constant-time implementation strategies.

## 6. **Power-of-two arithmetic.**

The native modulus of *Viper* is a power of two, allowing compression, decompression, and quantization to be implemented using fixed-latency integer operations. This arithmetic structure is particularly attractive for constrained embedded devices and hardware accelerators, where minimizing control logic and modular-reduction circuitry is often desirable.

## 7. **Uniform parameter scaling.**

All parameter sets share the same algorithmic framework and differ only in a small number of parameters. This uniformity simplifies implementation maintenance, testing, and security evaluation. It also facilitates future parameter adjustments without requiring changes to the underlying protocol structure.

## 8. **Compatibility with existing deployment models.**

Although the underlying assumption differs from MLWE and MLWR, *Viper* follows the standard KEM abstraction and can be deployed using the same interfaces as existing post-quantum key-establishment mechanisms. Migration from existing lattice-based infrastructures therefore requires relatively limited engineering effort.

## 7.2 **Limitations**

### 1. **Additional dither-generation overhead.**

Public dithers must be generated deterministically from shared seeds during encryption and decryption. Although the computational cost of this process is modest, it introduces additional XOF invocations that are absent in some alternative constructions. The resulting overhead is typically small but should nevertheless be taken into account in highly constrained environments.

## 2. NTT optimization is less straightforward.

The power-of-two modulus that simplifies quantization also prevents the direct use of the standard length-256 negacyclic NTT employed by many MLWE implementations. High-performance implementations therefore rely on auxiliary moduli and CRT reconstruction techniques. While these methods are well understood, they introduce additional engineering complexity compared with native NTT-friendly parameter choices.

## 3. Limited deployment experience at higher security levels.

The 384-bit and 512-bit parameter sets included in this specification demonstrate the scalability of the framework. However, these parameter sets have not yet received the same level of implementation testing and performance evaluation as the primary 128-bit, 192-bit, and 256-bit configurations. Further practical experience will help refine their optimization and deployment characteristics.

# 8 Implementation Specification

This section specifies the engineering conventions required for interoperable Viper implementations. The algorithms in the previous sections define the mathematical operations. This section fixes the corresponding implementation boundary, byte layouts, integer representation, expansion rules, sampling order, quantization rules, message encoding, and constant-time decapsulation behavior.

The submitted `API_PKC` package is the formal implementation boundary. It contains one Reference instance and one Optimized instance for each submitted profile. The Reference implementation is the portability baseline. Optimized implementations may use different internal layouts, vectorized arithmetic, precomputation, or auxiliary multiplication techniques, but they must produce identical public keys, ciphertexts, shared secrets, and Known Answer Test outputs for the same inputs.

All external objects are byte strings. Polynomial coefficients are serialized in increasing coefficient order. Module vectors are serialized in increasing component order. Unless explicitly stated otherwise, all profile-dependent byte lengths are the declared API lengths returned by the corresponding `kem_get_*` length-query functions and are listed in Table 10.

## 8.1 External API and Implementation Boundary

For each submitted profile, the formal implementation exposes the NGCC `API_PKC` KEM interface: `kem_keygen`, `kem_enc`, `kem_dec`. The corresponding length-query functions are `kem_get_pk_len_bytes`, `kem_get_sk_len_bytes`, `kem_get_ss_len_bytes`, `kem_get_ct_len_bytes`. They return the profile constants  $|pk|$ ,  $|sk|$ ,  $|ss|$ ,  $|ct|$ .

The submitted instances are organized as follows:

`API_PKC/Implementations/Reference_Implementation/MAMBA-Viper-{128,192,256,384,512}/`, and `API_PKC/Implementations/Optimized_Implementation/MAMBA-Viper-{128,192,256,384,512}/`. The directory `API_PKC/Implementations/Reference_Implementation/AlgorithmInstance/` contains the official NGCC support files. The files `KAT_KEM.c`, `drng.c/drng.h`, and `auxfunc.c/auxfunc.h` are copied unchanged into each submitted instance.

The implementation uses a common macro-instantiated core. Each profile fixes  $n$ ,  $q$ ,  $k$ ,  $\eta_s$ ,  $\eta_r$ ,  $t_{pk}$ ,  $t_u$ ,  $t_v$ ,  $|m|$ ,  $|ss|$ , and then instantiates the same KEM flow. The submitted profiles use  $n = 256$ ,  $q = 2^{12}$  for Viper-128, Viper-192, and Viper-256, and  $q = 2^{13}$  for Viper-384 and Viper-512.

## 8.2 Integer Representation

All polynomial coefficients are represented internally as unsigned 16-bit words. Since  $q \leq 2^{13}$  for all submitted profiles, this representation covers all coefficient values. Reduction modulo  $q$  is implemented by masking with  $q - 1$ . The centered interpretation is used only for analysis, decoding, or signed intermediate reasoning. Serialized values use the canonical nonnegative representation.

For  $x \in \{\mathbf{pk}, \mathbf{u}, \mathbf{v}\}$ , set  $p_x = 2^{t_x}$ ,  $\Delta_x = q/p_x$ ,  $g_x = \log_2 \Delta_x = \log_2 q - t_x$ . The implementation uses shifts for multiplication and division by  $\Delta_x$ . No general modular reduction, modular inversion, or rejection loop is required by the power-of-two arithmetic layer.

## 8.3 Serialized Objects

The public key consists of a 32-byte public seed  $\rho$ , followed by the packed public-key quantization label:

$$\mathbf{pk} = \rho \parallel \text{Pack}_{t_{\mathbf{pk}}}(\mathbf{b}), \quad |\text{Pack}_{t_{\mathbf{pk}}}(\mathbf{b})| = \frac{kn t_{\mathbf{pk}}}{8}.$$

Here  $\mathbf{b} \in R_{p_{\mathbf{pk}}}^k$  is the split public-key label.

The ciphertext consists of the packed  $\mathbf{u}$  component, the packed  $v$  component, and the 32-byte public dither seed  $\mu$ :

$$\mathbf{ct} = \text{Pack}_{t_{\mathbf{u}}}(\mathbf{u}) \parallel \text{Pack}_{t_v}(v) \parallel \mu,$$

where

$$|\text{Pack}_{t_{\mathbf{u}}}(\mathbf{u})| = \frac{kn t_{\mathbf{u}}}{8}, \quad |\text{Pack}_{t_v}(v)| = \frac{nt_v}{8}.$$

The KEM secret key stores the expanded PKE secret vector, the public key, the public-key hash, and the fallback string:

$$\mathbf{sk} = \text{Encode}_s(\mathbf{s}) \parallel \mathbf{pk} \parallel h_{\mathbf{pk}} \parallel z.$$

The expanded PKE secret length is

$$|\text{Encode}_s(\mathbf{s})| = \frac{kn \log_2 q}{8}.$$

The public-key hash has length 32 bytes, and the fallback string has length  $|z| = |\mathbf{ss}|$ . The fallback string is used only in implicit rejection.

Table 12: Serialized object layout in the submitted Viper implementation.

| Object        | Layout                                                                                    |
|---------------|-------------------------------------------------------------------------------------------|
| $\mathbf{pk}$ | $\rho \parallel \text{Pack}_{t_{\mathbf{pk}}}(\mathbf{b})$                                |
| $\mathbf{ct}$ | $\text{Pack}_{t_{\mathbf{u}}}(\mathbf{u}) \parallel \text{Pack}_{t_v}(v) \parallel \mu$   |
| $\mathbf{sk}$ | $\text{Encode}_s(\mathbf{s}) \parallel \mathbf{pk} \parallel h_{\mathbf{pk}} \parallel z$ |
| $\mathbf{ss}$ | $ \mathbf{ss} $ -byte shared secret                                                       |

## 8.4 Bit Packing and Parsing

For a sequence of coefficients  $(x_0, \dots, x_{L-1})$ ,  $\text{Pack}_t$  serializes the  $t$  least significant bits of each coefficient. Coefficients are processed in increasing index order. The implementation fills output bytes by a fixed accumulator-based rule, and the inverse function  $\text{Unpack}_t$  reads the same bit stream and reconstructs coefficients in  $\{0, \dots, 2^t - 1\}$ .

This convention applies uniformly to the packed public-key label  $\mathbf{b}$ , the ciphertext-vector label  $\mathbf{u}$ , and the scalar ciphertext label  $v$ . Unused output coefficients, if any, are set to zero before parsing. Optimized pack/unpack routines may group multiple coefficients per loop iteration, but they must implement the same external byte format.

## 8.5 Seed Expansion and Public Dithers

The public seed  $\rho$  determines both the public matrix  $\mathbf{A}$  and the public-key dither  $\mathbf{d}_{\text{pk}}$ . The ciphertext seed  $\mu$  determines the ciphertext-side dithers  $\mathbf{d}_{\text{u}}$  and  $d_{\text{v}}$ . All expansion calls are domain separated by component labels and indices.

For the public matrix, the expanded stream is parsed into coefficients modulo  $q$ . For a dither component  $x \in \{\text{pk}, \text{u}, \text{v}\}$ , the expanded stream is parsed into coefficients in  $\{0, \dots, \Delta_x - 1\} = \{0, \dots, 2^{g_x} - 1\}$ . Equivalently,  $g_x$  bits are consumed per dither coefficient.

Table 13: Dithered quantization inputs in the submitted Viper implementation.

| Component | Width           | Dither seed |
|-----------|-----------------|-------------|
| <b>b</b>  | $t_{\text{pk}}$ | $\rho$      |
| <b>u</b>  | $t_{\text{u}}$  | $\mu$       |
| $v$       | $t_{\text{v}}$  | $\mu$       |

The formal `API_PKC` implementation routes hash, XOF, and randomness expansion through the official auxiliary-function interface. The functions `sm3hash`, `pseudohash`, and `pseudoXOF` provide the submitted hash/XOF backend, and `drng.c/drng.h` provide the official deterministic random-number generator used by the KAT framework and API tests.

## 8.6 Secret Sampling

Long-term and ephemeral secrets are sampled from centered binomial distributions. For  $\mathcal{B}_\eta$ , the implementation expands a seed into a fixed bit stream and maps each coefficient to

$$\sum_{j=0}^{\eta-1} b_j - \sum_{j=0}^{\eta-1} b'_j, \quad b_j, b'_j \in \{0, 1\}.$$

The result lies in  $\{-\eta, \dots, \eta\}$  and is stored modulo  $q$  in the unsigned internal representation.

The long-term secret vector  $\mathbf{s}$  uses  $\eta_s$ . The ephemeral secret vector  $\mathbf{r}$  uses  $\eta_r$ . Coefficients are filled in increasing polynomial index order and increasing module-component order.

## 8.7 Dithered Quantization and Compensation

Let  $x \in \{\text{pk}, \text{u}, \text{v}\}$ , and let  $t_x$ ,  $\Delta_x$ , and  $g_x$  be the corresponding compression width, quantization step, and shift value. For an input coefficient  $a \in \mathbb{Z}_q$  and a dither coefficient  $d \in \{0, \dots, \Delta_x - 1\}$ , the transmitted label is

$$b = \mathbf{Q}_{\Delta_x \mathbb{Z}, d}(a) / \Delta_x = ((a + d + 2^{g_x-1}) \ggg g_x) \& (2^{t_x} - 1).$$

Thus  $b \in \mathbb{Z}_{p_x}$  is the compact split coefficient stored by the implementation. The compensated normal-form coefficient is reconstructed as

$$\hat{a} = \Delta_x b - d = ((b \lll g_x) - d) \& (q - 1).$$

This rule is used for all quantized components:

$$\hat{\mathbf{b}} = \Delta_{\text{pk}} \mathbf{b} - \mathbf{d}_{\text{pk}}, \quad \hat{\mathbf{u}} = \Delta_{\text{u}} \mathbf{u} - \mathbf{d}_{\text{u}}, \quad \hat{v} = \Delta_{\text{v}} v - d_{\text{v}},$$

coefficientwise modulo  $q$ . The serialized objects store the split labels  $\mathbf{b}$ ,  $\mathbf{u}$ , and  $v$ , while arithmetic and decryption use the compensated normal forms.

## 8.8 $E_8$ Labeling and Delabeling

This subsection specifies the public  $E_8$  labeling and delabeling maps used by the message codec in Section 2.2. The maps are fixed by the profile parameter  $b_{\text{msg}} = \text{VIPER\_MSG\_BITS\_PER\_COEFF}$ . Set  $M = 2^{b_{\text{msg}}}$  and  $\alpha_{E_8} = q/M$ . For one active 8-dimensional message block, the mixed-radix coordinate widths are

$$\beta = (\beta_0, \dots, \beta_7) = (b_{\text{msg}} - 1, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}} + 1).$$

Let  $p_i = 2^{\beta_i}$ . Equivalently,  $(p_0, \dots, p_7) = (M/2, M, M, M, M, M, M, 2M)$ , with  $M/2 = 1$  when  $b_{\text{msg}} = 1$ . The mixed-radix index set is

$$\mathcal{I}_{E_8, M} = \prod_{i=0}^7 \{0, \dots, p_i - 1\}.$$

It has size  $\prod_{i=0}^7 p_i = M^8$ , which is the size of the quotient  $E_8/(M\mathbb{Z}^8)$ .

We use the following rectangular-form basis of  $E_8$ :

$$\mathbf{B}_{E_8} = \begin{pmatrix} 2 & -1 & 0 & 0 & 0 & 0 & 0 & 1/2 \\ 0 & 1 & -1 & 0 & 0 & 0 & 0 & 1/2 \\ 0 & 0 & 1 & -1 & 0 & 0 & 0 & 1/2 \\ 0 & 0 & 0 & 1 & -1 & 0 & 0 & 1/2 \\ 0 & 0 & 0 & 0 & 1 & -1 & 0 & 1/2 \\ 0 & 0 & 0 & 0 & 0 & 1 & -1 & 1/2 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1/2 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1/2 \end{pmatrix}.$$

This basis has rectangular form  $\mathbf{B}_{E_8} = \mathbf{U} \text{diag}(2, 1, 1, 1, 1, 1, 1, 1/2)$  with  $\mathbf{U} \in \text{GL}_8(\mathbb{Z})$ . Hence  $M\mathbb{Z}^8 = \mathbf{B}_{E_8} \text{diag}(p_0, \dots, p_7)\mathbb{Z}^8$ , and the mixed-radix index set  $\mathcal{I}_{E_8, M}$  labels all cosets of  $E_8/(M\mathbb{Z}^8)$ .

---

### Algorithm 9 $\text{Label}_{E_8, b_{\text{msg}}}(a_0, \dots, a_7)$

---

**Require:** Integers  $a_i \in \{0, \dots, p_i - 1\}$ , where  $p_i = 2^{\beta_i}$ .

**Ensure:** Label vector  $\lambda \in E_8/(M\mathbb{Z}^8)$ .

- 1:  $\mathbf{a} \leftarrow (a_0, \dots, a_7)^\top$ .
  - 2:  $\lambda \leftarrow \mathbf{B}_{E_8} \mathbf{a} \bmod M\mathbb{Z}^8$ .
  - 3: **return**  $\lambda$ .
- 

Reduction modulo  $M\mathbb{Z}^8$  means choosing the fixed representative of the coset in  $E_8/(M\mathbb{Z}^8)$ . The map  $\text{Label}_{E_8, b_{\text{msg}}}$  is a public bijection from  $\mathcal{I}_{E_8, M}$  to  $E_8/(M\mathbb{Z}^8)$ . When this label is embedded into a coefficient block of  $R_q$ , the stored representative is  $\alpha_{E_8} \lambda \bmod q\mathbb{Z}^8$ .

---

### Algorithm 10 $\text{Delabel}_{E_8, b_{\text{msg}}}(\lambda)$

---

**Require:** Label vector  $\lambda \in E_8/(M\mathbb{Z}^8)$ .

**Ensure:** Integers  $(a_0, \dots, a_7) \in \mathcal{I}_{E_8, M}$ .

- 1:  $\mathbf{a}^* \leftarrow \mathbf{B}_{E_8}^{-1} \lambda$ .
  - 2: **for**  $i = 0$  to 7 **do**
  - 3:    $a_i \leftarrow (\mathbf{a}^*)_i \bmod p_i$ , with representative in  $\{0, \dots, p_i - 1\}$ .
  - 4: **end for**
  - 5: **return**  $(a_0, \dots, a_7)$ .
- 

The inverse map is taken in the quotient coordinates determined by  $\mathbf{B}_{E_8}$ . For every valid decoded label  $\lambda$ , the vector  $\mathbf{B}_{E_8}^{-1} \lambda$  is integral modulo  $(p_0, \dots, p_7)$ , and  $\text{Delabel}_{E_8, b_{\text{msg}}}$  recovers exactly

the mixed-radix block used by  $\text{Label}_{E_8, b_{\text{msg}}}$ . The division by  $\alpha_{E_8}$  that appears before delabeling in message decoding is a coefficient-domain rescaling of a decoded lattice point from  $\alpha_{E_8} E_8 / q \mathbb{Z}^8$  to  $E_8 / (M \mathbb{Z}^8)$ , not division in  $R_q$ .

## 8.9 Polynomial Arithmetic

The core ring arithmetic is over  $R_q = \mathbb{Z}_q[X] / (X^{256} + 1)$ . The reference implementation uses portable C polynomial multiplication and module operations. The optimized x86 implementation may use AVX2 vectorization, Toom–Cook/Karatsuba multiplication, or auxiliary-prime NTT multiplication followed by CRT reconstruction. These choices are internal backend choices. They preserve the same ring semantics, byte encodings, KAT outputs, and external API.

The implementation exposes the following abstract operations to the KEM layer: **PolyMul**, **MatVec**, **MatTVec**, **Dot**. They compute negacyclic polynomial products, **As**,  $\mathbf{A}^\top \mathbf{r}$ , and module inner products, respectively.

## 8.10 Constant-Time Decapsulation

Decapsulation always returns a shared secret for valid-length API inputs. The implementation performs decryption, message decoding, deterministic re-encryption, ciphertext comparison, and key selection without branching on secret data or on ciphertext validity.

The ciphertext comparison scans the full ciphertext. If the recomputed ciphertext matches the received ciphertext, the selected key material is the FO pre-key. Otherwise, the selected key material is the fallback string  $z$ . The final shared secret is computed from the selected key material and the ciphertext hash. The selection is implemented by a mask or conditional move.

Thus invalid ciphertexts are handled by implicit rejection. They do not produce an externally visible decryption-failure symbol, and they do not change the API return convention except for ordinary API-level errors such as invalid input lengths or null pointers.

## 8.11 Validation Requirements

A conforming implementation must satisfy the following requirements. The serialized **pk**, **ct**, **sk**, and **ss** byte strings must have exactly the declared profile lengths. Packing and unpacking must be bit-compatible with the Reference implementation. Seed expansion, CBD sampling, dither generation, quantization, compensation, message encoding, and message decoding must be deterministic and match the Reference output for the same inputs. Reference and Optimized instances must pass the same official **API\_PKC** KATs. Optimized implementations must preserve the same public API, byte layout, and implicit-rejection behavior as the Reference implementation.

## A Mathematical Foundations

This section presents the hardness assumptions underlying *Viper*. We recall MLWE [LS15] and extend LWQ [LLL24] to the module setting, leading to the definitions of MLWQ and Split-MLWQ below.

**Definition A.1** (MLWE). *Let  $k, m, q \in \mathbb{N}$ , and let  $\chi_s, \chi_e$  be distributions over  $R_q$ . The MLWE distribution  $\text{MLWE}_{q, \chi_s, \chi_e}$  is defined by  $\mathbf{A} \leftarrow \mathcal{U}(R_q^{m \times k})$ ,  $\mathbf{s} \leftarrow \chi_s^k$ ,  $\mathbf{e} \leftarrow \chi_e^m$ , and  $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$ . The output is  $(\mathbf{A}, \mathbf{b})$ . Computational problems on MLWE are defined as follows.*

1. **Search-MLWE.** *Given samples from  $\text{MLWE}_{q, \chi_s, \chi_e}$ , recover  $\mathbf{s}$ .*
2. **Decision-MLWE.** *Distinguish  $\text{MLWE}_{q, \chi_s, \chi_e}$  from the uniform distribution over  $R_q^{m \times k} \times R_q^m$ .*

**Definition A.2** (MLWQ). *Let  $k, m, q \in \mathbb{N}$ , let  $\chi_s$  be a distribution over  $R_q$ , let  $\Lambda \subset \mathbb{R}^n$  be a lattice, and let  $\mathcal{S} \subset R$  denote the polynomial dither domain. The MLWQ distribution  $\text{MLWQ}_{q, \chi_s, \Lambda}$  is defined by  $\mathbf{A} \leftarrow \mathcal{U}(R_q^{m \times k})$ ,  $\mathbf{s} \leftarrow \chi_s^k$ ,  $\mathbf{d} \leftarrow \mathcal{U}(\mathcal{S}^m)$ , and  $\mathbf{b} = \mathbf{Q}_{\Lambda, \mathbf{d}}^\diamond(\mathbf{A}\mathbf{s}) - \mathbf{d}$ . The output is  $(\mathbf{A}, \mathbf{b})$ . Computational problems on MLWQ are defined as follows.*

1. **Search-MLWQ.** *Given samples from  $\text{MLWQ}_{q, \chi_s, \Lambda}$ , recover  $\mathbf{s}$ .*
2. **Decision-MLWQ.** *Distinguish  $\text{MLWQ}_{q, \chi_s, \Lambda}$  from the uniform distribution over  $R_q^{m \times k} \times R_q^m$ .*

**Definition A.3** (Split-MLWQ). *Let  $k, m, q \in \mathbb{N}$ , let  $\chi_s$  be a distribution over  $R_q$ , let  $\Lambda \subset \mathbb{R}^n$  be a lattice, and let  $\mathcal{S} \subset R$  denote the polynomial dither domain. The Split-MLWQ distribution  $\text{SMLWQ}_{q, \chi_s, \Lambda}$  is defined by  $\mathbf{A} \leftarrow \mathcal{U}(R_q^{m \times k})$ ,  $\mathbf{s} \leftarrow \chi_s^k$ ,  $\mathbf{d} \leftarrow \mathcal{U}(\mathcal{S}^m)$ , and  $\mathbf{c} = \mathbf{Q}_{\Lambda, \mathbf{d}}^\diamond(\mathbf{A}\mathbf{s})$ . The output is  $(\mathbf{A}, \mathbf{c}, \mathbf{d})$ . Computational problems on Split-MLWQ are defined as follows.*

1. **Search Split-MLWQ.** *Given samples from  $\text{SMLWQ}_{q, \chi_s, \Lambda}$ , recover  $\mathbf{s}$ .*
2. **Decision Split-MLWQ.** *Distinguish  $\text{SMLWQ}_{q, \chi_s, \Lambda}$  from the uniform distribution over  $R_q^{m \times k} \times (\tau^{-1}(\Lambda \cap \mathbb{Z}_q^n))^m \times \mathcal{S}^m$ .*

*Viper* uses the split MLWQ problem because the dither is public. The normal form is used for security analysis.

**Theorem A.4.** *Let  $q = 2^\ell$  and  $p = 2^t$  with  $t \leq \ell$ , so  $\Delta = q/p = 2^{\ell-t}$ . Let  $\Lambda = \Delta\mathbb{Z}^n$ . Let  $d$  be uniform over  $\{0, \dots, \Delta - 1\}$ . For any fixed  $y \in \mathbb{Z}_q$ , define  $e = \mathbf{Q}_{\Delta\mathbb{Z}, d}(y) - d - y$ . Then  $e$  is uniform over  $\{-\Delta/2 + 1, \dots, \Delta/2\}$ , with mean  $1/2$  and variance  $(\Delta^2 - 1)/12$ , and this distribution is independent of  $y$ .*

*Proof.* Because  $d$  is uniform over  $\{0, \dots, \Delta - 1\}$  and independent of  $y$ , the value  $y + d$  is uniformly distributed over a complete interval of length  $\Delta$ . Within this interval, the nearest multiple of  $\Delta$  is fixed, and the error  $e$  ranges over exactly  $\{-\Delta/2 + 1, \dots, \Delta/2\}$  with equal probability  $1/\Delta$ . This distribution depends only on  $\Delta$ , not on  $y$ .  $\square$

**Corollary A.5** (Input-Independent Vector Error). *Let  $\mathbf{y} \in R_q^m$  and  $\mathbf{d} \in \mathcal{D}^m$  where  $\mathcal{D}$  samples each coefficient uniformly from  $\{0, \dots, \Delta - 1\}$  independently. Then the coefficient-wise quantization error  $\mathbf{e} = \mathbf{Q}_{\Lambda, \mathbf{d}}^\diamond(\mathbf{y}) - \mathbf{d} - \mathbf{y}$  has independent coefficients, each distributed as in Theorem A.4, and this distribution is independent of  $\mathbf{y}$ .*

We work in the divisible power-of-two setting. Let  $q = 2^\ell$ ,  $p = 2^t$ , and  $\Delta = q/p$ . Let  $D_\Delta \subset R_q$  be the set of polynomials whose coefficients lie in  $\{0, \dots, \Delta - 1\}$ , and let  $\Lambda = \Delta\mathbb{Z}^n$ . The scalar quantization-error distribution induced by uniform dithering is  $\chi_{q, p} = \mathcal{U}(\{-\Delta/2 + 1, \dots, \Delta/2\})$ .

We first consider the normal form. For  $\mathbf{y} \in R_q^m$  and  $\mathbf{d} \leftarrow \mathcal{U}(D_\Delta^m)$ , define the compensated quantization error as  $\mathbf{e} = \mathbf{Q}_{\Lambda, \mathbf{d}}^\diamond(\mathbf{y}) - \mathbf{d} - \mathbf{y}$ . By the dithered-quantization property, the coefficients of  $\mathbf{e}$  are independent of  $\mathbf{y}$  and follow  $\chi_{q, p}$ . Therefore, for  $\mathbf{y} = \mathbf{A}\mathbf{s}$ , a real normal-form MLWQ sample satisfies

$$\hat{\mathbf{b}} = \mathbf{Q}_{\Lambda, \mathbf{d}}^\diamond(\mathbf{A}\mathbf{s}) - \mathbf{d} = \mathbf{A}\mathbf{s} + \mathbf{e}, \quad \mathbf{e} \leftarrow \chi_{q, p}^m,$$

where  $\mathbf{e}$  is independent of  $(\mathbf{A}, \mathbf{s})$ .

**Theorem A.6.** *For every PPT distinguisher  $\mathcal{A}$  against normal-form MLWQ with parameters  $(q, k, m, p, \chi_s)$ , there exists a PPT distinguisher  $\mathcal{B}$  against decisional MLWE with secret distribution  $\chi_s$  and coefficient error distribution  $\chi_{q,p}$  such that*

$$\text{Adv}_{q,k,m,p,\chi_s}^{\text{Normal-MLWQ}}(\mathcal{A}) = \text{Adv}_{q,k,m,\chi_s,\chi_{q,p}}^{\text{MLWE}}(\mathcal{B}).$$

*In fact, the reduction is the identity map on normal-form samples.*

*Proof.* A real normal-form MLWQ sample is  $(\mathbf{A}, \hat{\mathbf{b}})$ , where  $\mathbf{A} \leftarrow U(R_q^{m \times k})$ ,  $\mathbf{s} \leftarrow \chi_s^k$ , and  $\hat{\mathbf{b}} = \mathbf{A}\mathbf{s} + \mathbf{e}$  with  $\mathbf{e} \leftarrow \chi_{q,p}^m$  independent of  $(\mathbf{A}, \mathbf{s})$ . This is exactly the real MLWE distribution with secret distribution  $\chi_s$  and error distribution  $\chi_{q,p}$ . The random normal-form MLWQ distribution is uniform over  $R_q^{m \times k} \times R_q^m$ , which is exactly the random branch of decisional MLWE. Thus the identity map sends real samples to real samples and random samples to random samples with no statistical loss.  $\square$

We next relate the split representation to the normal representation. A split MLWQ sample consists of a public matrix  $\mathbf{A}$ , a quantization label  $\mathbf{c} \in R_p^m$ , and a public dither  $\mathbf{d} \in D_\Delta^m$ . Its normal form is obtained by the public transformation  $(\mathbf{A}, \mathbf{c}, \mathbf{d}) \mapsto (\mathbf{A}, \hat{\mathbf{b}})$ , where  $\hat{\mathbf{b}} = \Delta\mathbf{c} - \mathbf{d} \in R_q^m$ . Conversely, every coefficient  $\hat{b} \in \mathbb{Z}_q$  has a unique decomposition  $\hat{b} = \Delta c - d$  with  $c \in \mathbb{Z}_p$  and  $d \in \{0, \dots, \Delta - 1\}$ . Indeed,  $d$  is the unique element satisfying  $d \equiv -\hat{b} \pmod{\Delta}$ , and then  $c = (\hat{b} + d)/\Delta \pmod{p}$ .

**Theorem A.7.** *For every PPT distinguisher  $\mathcal{A}$  against decisional Split-MLWQ with parameters  $(q, k, m, p, \chi_s)$ , there exists a PPT distinguisher  $\mathcal{B}$  against the corresponding normal-form MLWQ distribution such that*

$$\text{Adv}_{q,k,m,p,\chi_s}^{\text{Split-MLWQ}}(\mathcal{A}) = \text{Adv}_{q,k,m,p,\chi_s}^{\text{Normal-MLWQ}}(\mathcal{B}).$$

*The same statement also holds in the reverse direction.*

*Proof.* The reduction from split form to normal form applies the public map  $(\mathbf{c}, \mathbf{d}) \mapsto \Delta\mathbf{c} - \mathbf{d}$ . A real split sample is mapped to the corresponding real normal-form sample by construction. A uniform split sample has  $\mathbf{c} \leftarrow U(R_p^m)$  and  $\mathbf{d} \leftarrow U(D_\Delta^m)$ , and the coefficient-wise bijection  $(c, d) \mapsto \Delta c - d$  maps it to a uniform element of  $R_q^m$ . Thus real samples and random samples are both preserved exactly.

For the reverse direction, the reduction receives  $(\mathbf{A}, \hat{\mathbf{b}})$ , applies the coefficient-wise inverse decomposition  $\hat{b} = \Delta c - d$ , and gives  $(\mathbf{A}, \mathbf{c}, \mathbf{d})$  to the split-form distinguisher. The same bijection maps real normal-form samples to real split samples and uniform normal-form samples to uniform split samples. Therefore the distinguishing advantage is preserved in both directions.  $\square$

Combining Theorems A.6 and A.7, the split MLWQ transcript used by *Viper* is decision-equivalent to an MLWE transcript with coefficient error distribution  $\chi_{q,p}$ . Consequently, the public-key and ciphertext-vector components of *Viper* can be analyzed through their compensated MLWE normal forms.

## B Decryption Failure Analysis

This appendix presents the discrete DFR computation utilized to derive the parameter table. We first detail the marginal-product estimator based on coordinate independence, and subsequently introduce a refined root-projection formulation that directly captures intra-block polynomial correlations, thereby providing an unconditionally rigorous bound.

The decryption noise is given by  $e_{\text{dec}} = \mathbf{e}_{\text{pk}}^\top \mathbf{r} - \mathbf{s}^\top \mathbf{e}_u + e_v$ , where  $\mathbf{s} \leftarrow \mathcal{B}_{\eta_s}^k$  and  $\mathbf{r} \leftarrow \mathcal{B}_{\eta_r}^k$ . The coefficient distributions of  $\mathbf{e}_{\text{pk}}$ ,  $\mathbf{e}_u$ , and  $e_v$  are drawn from  $\chi_{\text{pk}}$ ,  $\chi_u$ , and  $\chi_v$ , respectively. The  $E_8$  decoder operates on the centered coefficient vector  $\tau(e_{\text{dec}}) \in \mathbb{Z}^n$ . Recall that an active  $E_8$  block utilizes the fine decoding lattice  $\alpha E_8$  with  $\alpha = q/2^R$ , and the total number of active blocks is  $B_{E_8} = \ell_{\text{msg}}/(8R)$ .

**Discrete Marginal-Product Estimator.** Let  $\mathcal{D}_{\text{pk}}$  denote the distribution of the product  $e_{\text{pk},i}r_i$ , where  $e_{\text{pk},i} \leftarrow \chi_{\text{pk}}$  and  $r_i \leftarrow \mathcal{B}_{\eta_r}$ . Similarly, let  $\mathcal{D}_{\text{u}}$  be the distribution of  $-s_ie_{\text{u},i}$ , where  $s_i \leftarrow \mathcal{B}_{\eta_s}$  and  $e_{\text{u},i} \leftarrow \chi_{\text{u}}$ . For a single, fixed coefficient of  $e_{\text{dec}}$ , each of the two inner products contributes  $kn$  independent component products stemming from the  $k$  module dimension and the  $n$  ring degree. Thus, the exact scalar marginal law  $\mathcal{D}_E$  of one coefficient of  $e_{\text{dec}}$  is computed via

$$\mathcal{D}_E = \mathcal{D}_{\text{pk}}^{*(kn)} * \mathcal{D}_{\text{u}}^{*(kn)} * \chi_v,$$

where  $*$  denotes discrete convolution over  $\mathbb{Z}$ , and  $\mathcal{D}^{*a}$  represents the  $a$ -fold convolution. Equivalently, utilizing the probability generating function  $P_{\mathcal{D}}(z) = \sum_x \Pr[X = x]z^x$ , this scalar law is succinctly expressed as

$$P_E(z) = P_{\text{pk}}(z)^{kn} P_{\text{u}}(z)^{kn} P_{\chi_v}(z).$$

For a given  $E_8$  block, let  $\mathbf{e} = (E_1, \dots, E_8)$  where each  $E_i \leftarrow \mathcal{D}_E$  is treated independently. Let  $\mathcal{R}(E_8)$  denote the set of 240 Voronoi-relevant roots of  $E_8$ , normalized to squared norm 2. Correct decoding is guaranteed if  $\langle \mathbf{e}, \boldsymbol{\rho} \rangle < \alpha$  for every  $\boldsymbol{\rho} \in \mathcal{R}(E_8)$ . Consequently, the failure probability for a single block is bounded by the facet union bound:

$$\delta_{E_8} \leq \sum_{\boldsymbol{\rho} \in \mathcal{R}(E_8)} \Pr_{\mathbf{e} \leftarrow \mathcal{D}_E^{\otimes 8}} [\langle \mathbf{e}, \boldsymbol{\rho} \rangle \geq \alpha].$$

The roots of  $E_8$  naturally partition into pair roots and half roots. For the pair roots  $(\pm 1, \pm 1, 0, \dots, 0)$ , let  $E, E' \leftarrow \mathcal{D}_E$  be independent and define the tail probabilities

$$p_{++} = \Pr[E + E' \geq \alpha], \quad p_{+-} = \Pr[E - E' \geq \alpha], \quad p_{--} = \Pr[-E - E' \geq \alpha].$$

Accounting for the  $\binom{8}{2}$  coordinate pairs and four sign permutations per pair, the contribution of the pair roots is  $\delta_{\text{pair}} = \binom{8}{2}(p_{++} + 2p_{+-} + p_{--})$ .

For the half roots  $(\pm 1/2, \dots, \pm 1/2)$  containing an even number of minus signs, we evaluate the half-integer facet inequality by scaling it by a factor of 2. For  $m \in \{0, 2, 4, 6, 8\}$ , define

$$p_m = \Pr \left[ \sum_{i=1}^{8-m} E_i - \sum_{i=8-m+1}^8 E_i \geq 2\alpha \right],$$

where  $E_i \leftarrow \mathcal{D}_E$  independently. Since there are  $\binom{8}{m}$  half roots with exactly  $m$  negative signs, their contribution evaluates to  $\delta_{\text{half}} = \sum_{m \in \{0, 2, 4, 6, 8\}} \binom{8}{m} p_m$ .

Combining both root classes yields  $\delta_{E_8} \leq \delta_{\text{pair}} + \delta_{\text{half}}$ . Applying a union bound over all active message blocks produces the marginal-product estimator:

$$\delta_{\text{dec}}^{\text{indep}} \leq B_{E_8} \left[ \binom{8}{2} (p_{++} + 2p_{+-} + p_{--}) + \sum_{m \in \{0, 2, 4, 6, 8\}} \binom{8}{m} p_m \right].$$

The reported DFR values in Table 5 are computationally obtained from this exact formulation. Unlike continuous Gaussian approximations, this discrete evaluation entirely avoids approximation errors in the extreme tail. The preceding marginal-product estimator is highly efficient but inherently relies on the *independent-coordinate assumption*—assuming that the polynomial coefficients  $e_{\text{dec},i}$ , and consequently the 8 coordinates within an  $E_8$  block, are strictly independent. In reality, due to the anti-cyclic algebraic structure of the polynomial ring  $\mathbb{Z}_q[X]/(X^n + 1)$ , weak non-linear correlations exist among these coefficients.

In practice, modern lattice-based standards such as ML-KEM (Kyber) rely on this independence heuristic for parameter selection, compensating for any minor theoretical deviations by provisioning massive security margins. For instance, while the IND-CCA2 security baseline requires a DFR below  $2^{-128}$ , ML-KEM parameters are configured to yield theoretical failure rates significantly lower (e.g.,  $< 2^{-160}$ ), effortlessly absorbing any dependency-induced tail deviations.

**Correlation-Aware Root-Projection Form.** To achieve an unconditionally rigorous theoretical bound for our scheme, the following product-level formulation structurally eliminates this heuristic blind spot. By evaluating each  $E_8$  root projection directly from the negacyclic polynomial products, we explicitly capture the intra-block algebraic dependencies.

For an active  $E_8$  block  $j$ , define its corresponding noise vector as

$$\mathbf{e}_j = (\tau(e_{\text{dec}})_{8j}, \tau(e_{\text{dec}})_{8j+1}, \dots, \tau(e_{\text{dec}})_{8j+7}) \in \mathbb{Z}^8.$$

For any root  $\boldsymbol{\rho} \in \mathcal{R}(E_8)$ , let  $Z_{\boldsymbol{\rho},j} = \langle \mathbf{e}_j, \boldsymbol{\rho} \rangle$ . Denoting the block indices as  $I_j = \{8j, \dots, 8j+7\}$ , this projection linearly decomposes into  $Z_{\boldsymbol{\rho},j} = Z_{\boldsymbol{\rho},j}^{(\text{pk})} + Z_{\boldsymbol{\rho},j}^{(\text{u})} + Z_{\boldsymbol{\rho},j}^{(\text{v})}$ , where the constituent projections are

$$Z_{\boldsymbol{\rho},j}^{(\text{pk})} = \sum_{h=1}^k \langle (\mathbf{e}_{\text{pk},h} r_h)_{I_j}, \boldsymbol{\rho} \rangle, \quad Z_{\boldsymbol{\rho},j}^{(\text{u})} = - \sum_{h=1}^k \langle (s_h \mathbf{e}_{\text{u},h})_{I_j}, \boldsymbol{\rho} \rangle,$$

and  $Z_{\boldsymbol{\rho},j}^{(\text{v})} = \langle (e_{\text{v}})_{I_j}, \boldsymbol{\rho} \rangle$ .

Crucially, for a single negacyclic product  $ab \bmod (X^n + 1)$ , the projected component over the block  $I_j$  can be expanded as

$$\langle (ab)_{I_j}, \boldsymbol{\rho} \rangle = \sum_{t \in I_j} \rho_{t-8j} \sum_{i=0}^{n-1} \varepsilon_{t,i} a_i b_{\pi(t,i)},$$

where  $\pi(t,i) = (t-i) \bmod n$ , and  $\varepsilon_{t,i} \in \{\pm 1\}$  indicates the sign induced by the modulo reduction over  $X^n + 1$ . By factoring out the shared variable  $a_i$ , this can be equivalently rearranged to explicitly expose the dependency structure:

$$\langle (ab)_{I_j}, \boldsymbol{\rho} \rangle = \sum_{i=0}^{n-1} a_i \left( \sum_{t \in I_j} \rho_{t-8j} \varepsilon_{t,i} b_{\pi(t,i)} \right).$$

This exact arrangement demonstrates how a single coefficient variable  $a_i$  contributes simultaneously to multiple coordinates within the same  $E_8$  block, resolving the independence assumption.

Let  $\mathcal{L}_{\boldsymbol{\rho},j}^{(\text{pk},1)}$  be the exact law of  $\langle (\mathbf{e}_{\text{pk},1} r_1)_{I_j}, \boldsymbol{\rho} \rangle$ , let  $\mathcal{L}_{\boldsymbol{\rho},j}^{(\text{u},1)}$  be the law of  $-\langle (s_1 \mathbf{e}_{\text{u},1})_{I_j}, \boldsymbol{\rho} \rangle$ , and let  $\mathcal{L}_{\boldsymbol{\rho},j}^{(\text{v})}$  be the law of  $\langle (e_{\text{v}})_{I_j}, \boldsymbol{\rho} \rangle$ . Assuming independence strictly *across* the  $k$  module components (rather than within the ring), the exact projection law becomes

$$\mathcal{L}_{\boldsymbol{\rho},j} = \left( \mathcal{L}_{\boldsymbol{\rho},j}^{(\text{pk},1)} \right)^{*k} * \left( \mathcal{L}_{\boldsymbol{\rho},j}^{(\text{u},1)} \right)^{*k} * \mathcal{L}_{\boldsymbol{\rho},j}^{(\text{v})}.$$

The corresponding correlation-aware product-level DFR bound is thus

$$\delta_{\text{dec}}^{\text{corr}} \leq \sum_{j=0}^{B_{E_8}-1} \sum_{\boldsymbol{\rho} \in \mathcal{R}(E_8)} \Pr_{Z \leftarrow \mathcal{L}_{\boldsymbol{\rho},j}} [Z \geq \alpha].$$

## C Security Proof Details

This appendix provides the full security proof.

### C.1 IND-CPA Security

In Figure 1,  $\text{GenPublic}$ ,  $\text{GenDither}$ , and  $\text{GenSecret}_{\eta_r}$  denote the ideal expansion procedures used in the proof. The concrete XOF-based instantiation is connected to these games by the  $\mathbf{Adv}_{\text{XOF}}$  term in Theorem C.1.

| Game $G_0$ Real                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Game $G_1$ Random PK                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Game $G_2$ Random CT                                                                                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Setup</b><br>1. $\rho \leftarrow \mathcal{U}(\{0, 1\}^{256})$<br>2. $(\mathbf{A}, \mathbf{d}_{\text{pk}}) \leftarrow \text{GenPublic}(\rho)$<br>3. $\mathbf{s} \leftarrow \mathcal{B}_{\eta_s}^k$<br>4. $\mathbf{b} \leftarrow \mathcal{Q}_{\Lambda_{\text{pk}}, \mathbf{d}_{\text{pk}}}^\circ(\mathbf{A}\mathbf{s})/\Delta_{\text{pk}}$                                                                                                                                                                                                                 | <b>Setup</b><br>1. $\rho \leftarrow \mathcal{U}(\{0, 1\}^{256})$<br>2. $(\mathbf{A}, \mathbf{d}_{\text{pk}}) \leftarrow \text{GenPublic}(\rho)$<br>3. $\mathbf{b} \leftarrow \mathcal{U}(R_{p_{\text{pk}}}^k)$                                                                                                                                                                                                                                                                                                                                             | <b>Setup</b><br>1. $\rho \leftarrow \mathcal{U}(\{0, 1\}^{256})$<br>2. $(\mathbf{A}, \mathbf{d}_{\text{pk}}) \leftarrow \text{GenPublic}(\rho)$<br>3. $\mathbf{b} \leftarrow \mathcal{U}(R_{p_{\text{pk}}}^k)$                                   |
| <b>Challenge <math>m_b</math></b><br>5. $\sigma, \mu \leftarrow \mathcal{U}(\{0, 1\}^{256})$<br>6. $\mathbf{r} \leftarrow \text{GenSecret}_{\eta_r}(\sigma)$<br>7. $(\mathbf{d}_u, d_v) \leftarrow \text{GenDither}(\mu)$<br>8. $\hat{\mathbf{b}} \leftarrow \Delta_{\text{pk}}\mathbf{b} - \mathbf{d}_{\text{pk}}$<br>9. $\mathbf{u} \leftarrow \mathcal{Q}_{\Lambda_u, \mathbf{d}_u}^\circ(\mathbf{A}^\top \mathbf{r})/\Delta_u$<br>10. $v \leftarrow \mathcal{Q}_{\Lambda_v, d_v}^\circ(\hat{\mathbf{b}}^\top \mathbf{r} + \text{Encode}(m_b))/\Delta_v$ | <b>Challenge <math>m_b</math></b><br>4. $\sigma, \mu \leftarrow \mathcal{U}(\{0, 1\}^{256})$<br>5. $\mathbf{r} \leftarrow \text{GenSecret}_{\eta_r}(\sigma)$<br>6. $(\mathbf{d}_u, d_v) \leftarrow \text{GenDither}(\mu)$<br>7. $\hat{\mathbf{b}} \leftarrow \Delta_{\text{pk}}\mathbf{b} - \mathbf{d}_{\text{pk}}$<br>8. $\mathbf{u} \leftarrow \mathcal{Q}_{\Lambda_u, \mathbf{d}_u}^\circ(\mathbf{A}^\top \mathbf{r})/\Delta_u$<br>9. $v \leftarrow \mathcal{Q}_{\Lambda_v, d_v}^\circ(\hat{\mathbf{b}}^\top \mathbf{r} + \text{Encode}(m_b))/\Delta_v$ | <b>Challenge <math>m_b</math></b><br>4. $\mu \leftarrow \mathcal{U}(\{0, 1\}^{256})$<br>5. $(\mathbf{d}_u, d_v) \leftarrow \text{GenDither}(\mu)$<br>6. $\mathbf{u} \leftarrow \mathcal{U}(R_{p_u}^k)$<br>7. $v \leftarrow \mathcal{U}(R_{p_v})$ |

Figure 1: Game-hopping sequence for the IND-CPA proof of Viper.PKE after replacing the concrete XOF-based expansion procedures by the ideal expansion experiment. The public-key hop replaces  $\mathbf{b}$  by a uniform element of  $R_{p_{\text{pk}}}^k$ . The ciphertext hop replaces  $\mathbf{u}$  and  $v$  jointly, because both real ciphertext components are generated from the same ephemeral secret  $\mathbf{r}$ .

**Theorem C.1** (IND-CPA Security). *For every PPT IND-CPA adversary  $\mathcal{A}$  against the seeded implementation of Viper.PKE, there exist PPT adversaries  $\mathcal{C}_{\text{XOF}}$ ,  $\mathcal{B}_1$ , and  $\mathcal{B}_2$  such that*

$$\begin{aligned} \text{Adv}_{\text{Viper.PKE}}^{\text{IND-CPA}}(\mathcal{A}) &\leq \text{Adv}_{\text{XOF}}(\mathcal{C}_{\text{XOF}}) + \text{Adv}_{q,k,k,\mathcal{B}_{\eta_s},\chi_{q,p_{\text{pk}}}}^{\text{MLWE}}(\mathcal{B}_1) \\ &\quad + \text{Adv}_{q,k,k+1,\mathcal{B}_{\eta_r},(\chi_{q,p_u}^k,\chi_{q,p_v})}^{\text{MLWE}}(\mathcal{B}_2). \end{aligned}$$

Here  $\text{Adv}_{\text{XOF}}$  denotes the distinguishing advantage for replacing the concrete domain-separated XOF-based expansion procedures by the ideal expansion experiment. The ideal experiment expands  $\rho$  to  $(\mathbf{A}, \mathbf{d}_{\text{pk}})$ , expands  $\mu$  to  $(\mathbf{d}_u, d_v)$ , and expands  $\sigma$  to  $\mathbf{r}$  with the prescribed output distributions.

*Proof.* Let  $S_{\text{real}}$  be the event that  $\mathcal{A}$  outputs the correct challenge bit in the seeded IND-CPA experiment. Let  $S_i$  be the event that  $\mathcal{A}$  outputs the correct challenge bit in Game  $G_i$  of Figure 1. The transition from the seeded experiment to Game  $G_0$  replaces the concrete XOF-based expansion procedures by the ideal expansion experiment. By the definition of  $\text{Adv}_{\text{XOF}}$ , there exists a PPT adversary  $\mathcal{C}_{\text{XOF}}$  such that

$$|\Pr[S_{\text{real}}] - \Pr[S_0]| \leq \text{Adv}_{\text{XOF}}(\mathcal{C}_{\text{XOF}}).$$

It remains to bound  $\Pr[S_0] - 1/2$  in the ideal expansion experiment.

**Game  $G_0$ .** This game is the IND-CPA experiment after the expansion replacement. In Games  $G_0$  and  $G_1$ , the challenge coins are sampled as independent  $\sigma, \mu \leftarrow \mathcal{U}(\{0, 1\}^{256})$ . The seed  $\sigma$  determines the ephemeral secret, and the seed  $\mu$  determines the ciphertext-side dithers.

**Game  $G_1$ .** We replace  $\mathbf{b}$  by an independent uniform vector in  $R_{p_{\text{pk}}}^k$ . In the real branch of Game  $G_0$ , the corresponding normal form satisfies  $\hat{\mathbf{b}} = \Delta_{\text{pk}}\mathbf{b} - \mathbf{d}_{\text{pk}} = \mathbf{A}\mathbf{s} + \mathbf{e}_{\text{pk}}$ , where the coefficients of  $\mathbf{e}_{\text{pk}}$  follow  $\chi_{q,p_{\text{pk}}}$ . Thus the real public-key normal-form sample is a decisional MLWE sample with secret distribution  $\mathcal{B}_{\eta_s}$  and error distribution  $\chi_{q,p_{\text{pk}}}$ . In Game  $G_1$ ,  $\mathbf{b}$  is uniform over  $R_{p_{\text{pk}}}^k$ , so  $\Delta_{\text{pk}}\mathbf{b}$  is uniform over  $(\tau^{-1}(\Lambda_{\text{pk}} \cap \mathbb{Z}_q^n))^k$ . By Theorem A.7, this is exactly the split representation of a uniform normal-form sample. Therefore there exists a PPT adversary  $\mathcal{B}_1$  such that

$$|\Pr[S_0] - \Pr[S_1]| \leq \text{Adv}_{q,k,k,\mathcal{B}_{\eta_s},\chi_{q,p_{\text{pk}}}}^{\text{MLWE}}(\mathcal{B}_1).$$

**Game  $G_2$ .** In Game  $G_1$ , the public-key normal form is independent of  $\mathbf{s}$ . We now replace  $\mathbf{u}$  and  $v$  jointly by independent uniform values in  $R_{p_u}^k$  and  $R_{p_v}$ . In the real branch, define  $\hat{\mathbf{u}} = \Delta_u\mathbf{u} - \mathbf{d}_u$  and  $\hat{v} = \Delta_v v - d_v$ . The stacked normal-form ciphertext satisfies

$$\begin{pmatrix} \hat{\mathbf{u}} \\ \hat{v} - \text{Encode}(m_b) \end{pmatrix} = \begin{pmatrix} \mathbf{A}^\top \\ \hat{\mathbf{b}}^\top \end{pmatrix} \mathbf{r} + \begin{pmatrix} \mathbf{e}_u \\ e_v \end{pmatrix}.$$

The upper  $k$  error rows follow  $\chi_{q,p_u}$ . The last error row follows  $\chi_{q,p_v}$ . The shift by  $\text{Encode}(m_b)$  in the last row is fixed after the challenge message is chosen and preserves both branches of the decisional experiment. Under the identification  $z \mapsto \Delta_x z$ , Theorem A.7 transfers the row-wise mixed-error decisional MLWE hop to the ciphertext distribution used in the game. Therefore there exists a PPT adversary  $\mathcal{B}_2$  such that

$$|\Pr[S_1] - \Pr[S_2]| \leq \text{Adv}_{q,k,k+1,\mathcal{B}_{\eta r},(\chi_{q,p_u}^k, \chi_{q,p_v})}^{\text{MLWE}}(\mathcal{B}_2).$$

In Game  $G_2$ , both  $\mathbf{u}$  and  $v$  are uniform and independent of  $m_b$ . Hence the challenge ciphertext is independent of the challenge bit, and  $\Pr[S_2] = 1/2$ . Combining the XOF replacement with the two MLWE hops gives the claimed bound.  $\square$

## C.2 IND-CCA Security

The FO proof applies once the underlying PKE is IND-CPA secure and the decryption-failure probability is negligible. We state the resulting bounds in the form used for parameter selection. The quantity  $\delta_{\text{dec}}$  denotes the whole-ciphertext decryption-failure probability from Section ?? . Let  $q_{\text{RO}} \triangleq q_{\mathcal{G}} + q_{\mathcal{H}}$ .

**Theorem C.2** (Classical ROM). *Assume that  $\mathcal{G}$  and  $\mathcal{H}$  are random oracles. For every classical adversary  $\mathcal{A}$  making at most  $q_{\mathcal{G}}$  queries to  $\mathcal{G}$ ,  $q_{\mathcal{H}}$  queries to  $\mathcal{H}$ , and  $q_D$  decapsulation queries, there exists an IND-CPA adversary  $\mathcal{B}$  against Viper.PKE such that*

$$\text{Adv}_{\text{Viper.KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq 3\text{Adv}_{\text{Viper.PKE}}^{\text{IND-CPA}}(\mathcal{B}) + (2q_{\text{RO}} + q_D)\delta_{\text{dec}} + \frac{3q_{\text{RO}}}{|\mathcal{M}_{\text{FO}}|}.$$

*Proof.* We use the concrete IND-CPA bound for the message-based implicit-rejection transform  $\text{KEM}_m^\chi$  given by Hofheinz, Hövelmanns, and Kiltz [HHK17, Section 3.3]. For this transform, the bound is

$$(2q_{\text{RO}} + q_D)\delta + \frac{3q_{\text{RO}}}{|\mathcal{M}|} + 3\text{Adv}_{\text{PKE}}^{\text{IND-CPA}}.$$

The scheme-specific condition is that encryption be deterministic once  $(\mathbf{pk}, m, \omega)$  is fixed. Here  $\omega$  determines both the ephemeral secret and the ciphertext-side dithers, while  $\rho$  in the public key determines  $\mathbf{A}$  and the public-key dither. The decapsulation algorithm decrypts, re-derives  $\omega'$ , re-encrypts, and applies implicit rejection. Thus Viper.KEM has the required message-based implicit-rejection FO form. Substituting  $\mathcal{M} = \mathcal{M}_{\text{FO}}$ ,  $\delta = \delta_{\text{dec}}$ , and  $q_{\text{RO}} = q_{\mathcal{G}} + q_{\mathcal{H}}$  gives the stated inequality.  $\square$

**Theorem C.3** (Quantum ROM). *Assume that  $\mathcal{G}$  and  $\mathcal{H}$  are quantum-accessible random oracles. For every quantum adversary  $\mathcal{A}$  against Viper.KEM making at most  $q_{\mathcal{G}}$  queries to  $\mathcal{G}$ ,  $q_{\mathcal{H}}$  queries to  $\mathcal{H}$ , and  $q_D$  classical decapsulation queries, there exists an IND-CPA adversary  $\mathcal{B}$  against Viper.PKE such that*

$$\text{Adv}_{\text{Viper.KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq \frac{2q_{\mathcal{H}}}{\sqrt{|\mathcal{M}_{\text{FO}}|}} + 4q_{\mathcal{G}}\sqrt{\delta_{\text{dec}}} + 2q_{\text{RO}}\sqrt{\text{Adv}_{\text{Viper.PKE}}^{\text{IND-CPA}}(\mathcal{B}) + \frac{1}{|\mathcal{M}_{\text{FO}}|}}.$$

*Proof.* We apply the implicit-rejection FO KEM bound in the quantum random-oracle model from Jiang, Zhang, Chen, Wang, and Ma [JZC<sup>+</sup>18, Theorem 1]. For a quantum adversary making  $q_{\mathcal{G}}$  queries to  $\mathcal{G}$ ,  $q_{\mathcal{H}}$  queries to  $\mathcal{H}$ , and  $q_D$  classical decapsulation queries, the theorem gives the loss

$$\frac{2q_{\mathcal{H}}}{\sqrt{|\mathcal{M}|}} + 4q_{\mathcal{G}}\sqrt{\delta} + 2(q_{\mathcal{G}} + q_{\mathcal{H}})\sqrt{\text{Adv}_{\text{PKE}}^{\text{OW-CPA}}}.$$

It remains only to express the one-way term through the IND-CPA security used in this paper. For uniformly sampled messages over  $\mathcal{M}$ , IND-CPA security implies

$$\text{Adv}_{\text{PKE}}^{\text{OW-CPA}} \leq \text{Adv}_{\text{PKE}}^{\text{IND-CPA}} + \frac{1}{|\mathcal{M}|}.$$

Indeed, an adversary that recovers the encrypted message with probability  $p$  gives a distinguisher between an encryption of that message and an encryption of an independent random message with advantage at least  $p - 1/|\mathcal{M}|$ .

The same determinism condition as in the classical proof holds. Once  $(\mathbf{pk}, m, \omega)$  is fixed, encryption is deterministic, because  $\omega$  fixes the ephemeral secret and the ciphertext-side dithers. The decapsulation algorithm decrypts, re-derives  $\omega'$ , re-encrypts, and applies implicit rejection. The only correctness loss is the event that decryption crosses the decoding boundary, which is accounted for by  $\delta_{\text{dec}}$ . Substituting  $\mathcal{M} = \mathcal{M}_{\text{FO}}$ ,  $\delta = \delta_{\text{dec}}$ , and  $q_{\text{RO}} = q_{\mathcal{G}} + q_{\mathcal{H}}$  gives the stated inequality.  $\square$

Together, Theorems C.2 and C.3 state the CCA security of **Viper.KEM** through the IND-CPA security of **Viper.PKE**. The KEM security bound is controlled by the decisional MLWE problem in Theorem C.1, the decryption-failure probability  $\delta_{\text{dec}}$ , and the size of the FO message space.

## D Additional Benchmark Results

This appendix reports native-interface x86 benchmark results for **Viper**. The formal `API_PKC` submission performance is reported in Section 4.3, where the measured entry points are the NGCC `API_PKC` functions `kem_keygen`, `kem_enc`, and `kem_dec`. The tables in this appendix use the native **Viper** benchmarking interface and the native low-level arithmetic and cryptographic components, matching the benchmarking layer used by the representative schemes in the comparison. This gives a like-for-like implementation comparison while separately preserving the formal `API_PKC` self-evaluation results in the main performance section.

### D.1 Native x86 Comparison to Other Lattice-Based KEMs

Table 14 compares **Viper** with representative lattice-based KEMs grouped by target security levels. For **Viper**, the efficiency entries are native-interface x86 measurements. An AVX2 entry  $a/b$  reports the Toom–Cook/Karatsuba backend and the auxiliary-prime NTT backend, respectively.

### D.2 Detailed Native x86 Breakdown

Table 15 gives the detailed native-backend arithmetic breakdown. These measurements use the native **Viper** benchmarking interface and expose the KEM, PKE, and arithmetic layers separately. The Toom–Cook/Karatsuba and auxiliary-prime NTT backends are shown separately for the AVX2 implementation.

Table 14: Native-interface comparison of *Viper* with representative KEMs grouped by target security levels. The Type column records the underlying problem family or structural setting. DFR denotes  $\log_2 \delta_{\text{dec}}$ . Efficiency is reported in kCycles. For *Viper*, the x86 entries report the native self-contained benchmarking interface, using the same benchmarking layer as the other schemes in the table. An AVX2 entry  $a/b$  reports the Toom–Cook/Karatsuba backend and the auxiliary-prime NTT backend, respectively. The formal API\_PKC self-evaluation numbers are reported separately in Section 4.3.

| Scheme                                         | Type  | q               | Sizes (bytes) |       |       | DFR  | Efficiency (kCycles) |       |       |         |         |         |
|------------------------------------------------|-------|-----------------|---------------|-------|-------|------|----------------------|-------|-------|---------|---------|---------|
|                                                |       |                 | pk            | ct    | sk    |      | Ref                  |       |       | AVX2    |         |         |
|                                                |       |                 |               |       |       |      | KG                   | Enc   | Dec   | KG      | Enc     | Dec     |
| 128-bit Security                               |       |                 |               |       |       |      |                      |       |       |         |         |         |
| Viper-128                                      | MLWQ  | 2 <sup>12</sup> | 608           | 736   | 1424  | -178 | 48                   | 67    | 70    | 33/33   | 48/48   | 47/47   |
| Kyber-512 [BDK <sup>+</sup> 18]                | MLWE  | 3329            | 800           | 768   | 1632  | -139 | 90                   | 107   | 157   | 23      | 24      | 21      |
| Π <sub>KEM-I</sub> [ZFY <sup>+</sup> 20]       | AMLWE | 7681            | 672           | 672   | 1568  | -82  | 160                  | 217   | 244   | 71      | 77      | 65      |
| LightSaber [DKRV18]                            | MLWR  | 2 <sup>13</sup> | 672           | 736   | 1568  | -120 | 56                   | 71    | 77    | 39/37   | 46/44   | 45/41   |
| Sable [MKKV21]                                 | MLWR  | 2 <sup>11</sup> | 608           | 672   | 800   | -139 | 49                   | 64    | 70    | 32      | 39      | 37      |
| LAC-128-v3b [LLZ <sup>+</sup> 18]              | RLWE  | 2 <sup>8</sup>  | 544           | 704   | 1056  | –    | 70                   | 119   | 158   | 18      | 27      | 33      |
| DAWN-α-512 [LZL <sup>+</sup> 25]               | NTRU  | 769             | 615           | 436   | 1319  | -133 | 92                   | 42    | 45    | 22      | 20      | 19      |
| DAWN-β-512 [LZL <sup>+</sup> 25]               | NTRU  | 257             | 514           | 450   | 1154  | -130 | 71                   | 35    | 48    | 23      | 22      | 23      |
| NEV-512 [ZFY23]                                | NTRU  | 769             | 615           | 615   | 1294  | -138 | 95                   | 88    | 113   | 21      | 34      | 26      |
| NEV'-512 [ZFY23]                               | NTRU  | 769             | 615           | 615   | 1294  | -200 | 89                   | 84    | 110   | 21      | 31      | 24      |
| Frost-128                                      | ULWQ  | 2 <sup>16</sup> | 5792          | 5856  | 7584  | -143 | 571                  | 764   | 863   | 483     | 690     | 807     |
| FrodoKEM-640 [BCD <sup>+</sup> 16]             | ULWE  | 2 <sup>15</sup> | 9616          | 9752  | 19888 | -138 | 1493                 | 7583  | 7629  | 1123    | 1271    | 1193    |
| SCloud <sup>+</sup> -128 [WZZ <sup>+</sup> 24] | ULWE  | 2 <sup>12</sup> | 7200          | 5456  | –     | -134 | 1177                 | 1215  | 1167  | 999     | 1062    | 1014    |
| 192-bit Security                               |       |                 |               |       |       |      |                      |       |       |         |         |         |
| Viper-192                                      | MLWQ  | 2 <sup>12</sup> | 992           | 1088  | 2200  | -172 | 86                   | 116   | 121   | 54/53   | 78/77   | 76/76   |
| Kyber-768 [BDK <sup>+</sup> 18]                | MLWE  | 3329            | 1184          | 1088  | 2400  | -164 | 160                  | 154   | 188   | 32      | 31      | 29      |
| Π <sub>KEM-II</sub> [ZFY <sup>+</sup> 20]      | AMLWE | 7681            | 896           | 992   | 2208  | -128 | 232                  | 295   | 332   | 90      | 100     | 86      |
| Saber [DKRV18]                                 | MLWR  | 2 <sup>13</sup> | 992           | 1088  | 2304  | -136 | 102                  | 127   | 152   | 69/60   | 75/70   | 73/67   |
| Sable [MKKV21]                                 | MLWR  | 2 <sup>11</sup> | 896           | 1024  | 1152  | -143 | 92                   | 119   | 127   | 54      | 65      | 63      |
| Espada [MKKV21]                                | MLWR  | 2 <sup>15</sup> | 1280          | 1304  | 1728  | -167 | 217                  | 234   | 225   | 184     | 196     | 190     |
| LAC-192-v3b [LLZ <sup>+</sup> 18]              | RLWE  | 2 <sup>8</sup>  | 1056          | 1352  | 1568  | –    | 118                  | 189   | 255   | 23      | 38      | 53      |
| Florete [MKKV21]                               | RLWR  | 2 <sup>10</sup> | 896           | 1248  | 1152  | -131 | 58                   | 96    | 124   | 37      | 58      | 66      |
| Frost-192                                      | ULWQ  | 2 <sup>16</sup> | 10680         | 10736 | 12680 | -234 | 1634                 | 1904  | 2076  | 1267    | 1704    | 1917    |
| FrodoKEM-976 [BCD <sup>+</sup> 16]             | ULWE  | 2 <sup>16</sup> | 15632         | 15792 | 31296 | -199 | 2945                 | 12823 | 12735 | 2290    | 2396    | 2255    |
| SCloud <sup>+</sup> -192 [WZZ <sup>+</sup> 24] | ULWE  | 2 <sup>12</sup> | 11136         | 10832 | –     | -201 | 2369                 | 2463  | 2384  | 2008    | 2040    | 1925    |
| 256-bit Security                               |       |                 |               |       |       |      |                      |       |       |         |         |         |
| Viper-256                                      | MLWQ  | 2 <sup>12</sup> | 1312          | 1472  | 2912  | -273 | 135                  | 174   | 179   | 70/68   | 102/99  | 98/96   |
| Kyber-1024 [BDK <sup>+</sup> 18]               | MLWE  | 3329            | 1568          | 1568  | 3168  | -174 | 237                  | 241   | 297   | 40      | 40      | 43      |
| Π <sub>KEM-III</sub> [ZFY <sup>+</sup> 20]     | AMLWE | 12289           | 1472          | 1536  | 3392  | -211 | 335                  | 461   | 506   | 146     | 175     | 141     |
| FireSaber [DKRV18]                             | MLWR  | 2 <sup>13</sup> | 1312          | 1472  | 3040  | -165 | 164                  | 196   | 212   | 95/88   | 110/102 | 109/100 |
| Sable [MKKV21]                                 | MLWR  | 2 <sup>11</sup> | 1312          | 1376  | 1632  | -208 | 157                  | 189   | 203   | 84      | 99      | 97      |
| LAC-256-v3b [LLZ <sup>+</sup> 18]              | RLWE  | 2 <sup>8</sup>  | 1056          | 1464  | 1824  | –    | 171                  | 306   | 454   | 30      | 55      | 106     |
| DAWN-α-1024 [LZL <sup>+</sup> 25]              | NTRU  | 769             | 1229          | 973   | 2605  | -163 | 138                  | 77    | 113   | 46      | 36      | 38      |
| DAWN-β-1024 [LZL <sup>+</sup> 25]              | NTRU  | 257             | 1027          | 1027  | 2275  | -138 | 135                  | 88    | 127   | 46      | 42      | 47      |
| NEV-1024 [ZFY23]                               | NTRU  | 769             | 1229          | 1229  | 2522  | -152 | 208                  | 184   | 257   | 38      | 64      | 51      |
| NEV'-1024 [ZFY23]                              | NTRU  | 769             | 1229          | 1229  | 2522  | -200 | 206                  | 172   | 251   | 38      | 60      | 46      |
| Frost-256                                      | ULWQ  | 2 <sup>16</sup> | 17192         | 17256 | 21216 | -290 | 2796                 | 3157  | 3452  | 2145    | 2832    | 3106    |
| FrodoKEM-1344 [BCD <sup>+</sup> 16]            | ULWE  | 2 <sup>16</sup> | 21520         | 21632 | 43088 | -252 | 5358                 | 29818 | 29988 | 4603    | 4551    | 3909    |
| SCloud <sup>+</sup> -256 [WZZ <sup>+</sup> 24] | ULWE  | 2 <sup>12</sup> | 18744         | 16916 | –     | -266 | 3826                 | 4236  | 4225  | 2953    | 3248    | 3150    |
| 384-bit Security                               |       |                 |               |       |       |      |                      |       |       |         |         |         |
| Viper-384                                      | MLWQ  | 2 <sup>13</sup> | 2496          | 2656  | 5488  | -352 | 366                  | 431   | 438   | 165/177 | 214/227 | 211/226 |
| Frost-384                                      | ULWQ  | 2 <sup>16</sup> | 25096         | 37736 | 29016 | -441 | 11387                | 7406  | 7892  | 3915    | 6213    | 6650    |
| 512-bit Security                               |       |                 |               |       |       |      |                      |       |       |         |         |         |
| Viper-512                                      | MLWQ  | 2 <sup>13</sup> | 3200          | 3456  | 7040  | -421 | 579                  | 677   | 684   | 248/246 | 312/313 | 314/307 |
| Frost-512                                      | ULWQ  | 2 <sup>16</sup> | 36432         | 72944 | 41696 | -581 | 21277                | 14673 | 15195 | 6805    | 12465   | 12902   |

Table 15: Native-backend Viper KEM, PKE, and arithmetic performance.

| Level | Impl. | Mult. | KEM    |        |        | PKE    |        |       | Arithmetic |        |         |               |               |
|-------|-------|-------|--------|--------|--------|--------|--------|-------|------------|--------|---------|---------------|---------------|
|       |       |       | KG     | Encaps | Decaps | KG     | Enc    | Dec   | Mul        | As     | $A^T r$ | $\hat{b}^T r$ | $s^T \hat{u}$ |
| 128   | Ref   | Toom  | 47766  | 67107  | 69502  | 31882  | 47272  | 15846 | 2660       | 10640  | 10640   | 5357          | 5319          |
|       | AVX2  | Toom  | 32718  | 48032  | 47424  | 17099  | 28842  | 10412 | 874        | 2850   | 2964    | 1899          | 1633          |
|       | AVX2  | NTT   | 32603  | 48145  | 47310  | 17252  | 28842  | 10184 | 1026       | 2812   | 2812    | 1709          | 1710          |
| 192   | Ref   | Toom  | 86108  | 116052 | 120649 | 67526  | 91200  | 19646 | 2736       | 24167  | 24205   | 8132          | 8132          |
|       | AVX2  | Toom  | 54036  | 78127  | 75962  | 35795  | 52174  | 14249 | 874        | 5928   | 5928    | 2204          | 2241          |
|       | AVX2  | NTT   | 52820  | 77178  | 75620  | 34466  | 50995  | 14097 | 1026       | 5358   | 5320    | 2394          | 2394          |
| 256   | Ref   | Toom  | 135470 | 174381 | 178752 | 114608 | 145008 | 24054 | 2660       | 42332  | 42408   | 10564         | 12464         |
|       | AVX2  | Toom  | 70490  | 101650 | 97850  | 50197  | 70870  | 16416 | 912        | 10107  | 10070   | 2887          | 2850          |
|       | AVX2  | NTT   | 68476  | 99484  | 96481  | 47727  | 69312  | 16568 | 1026       | 8550   | 8550    | 3078          | 3078          |
| 384   | Ref   | Toom  | 365978 | 431110 | 437646 | 334742 | 375060 | 39254 | 2697       | 129884 | 129732  | 18506         | 18506         |
|       | AVX2  | Toom  | 165148 | 214320 | 210900 | 148314 | 164236 | 25840 | 874        | 29412  | 29450   | 4788          | 4788          |
|       | AVX2  | NTT   | 177156 | 227050 | 225720 | 149796 | 178334 | 26562 | 1063       | 27398  | 27398   | 5738          | 5738          |
| 512   | Ref   | Toom  | 578511 | 677159 | 683999 | 541499 | 602603 | 49020 | 2660       | 214776 | 214548  | 23788         | 23788         |
|       | AVX2  | Toom  | 247722 | 311751 | 313842 | 214357 | 249545 | 31236 | 874        | 48260  | 48108   | 6156          | 6118          |
|       | AVX2  | NTT   | 245708 | 312815 | 306584 | 213180 | 248786 | 33098 | 1026       | 46778  | 46778   | 7638          | 7638          |

## References

- [ACC<sup>+</sup>22] Amin Abdulrahman, Jiun-Peng Chen, Yu-Jia Chen, Vincent Hwang, Matthias J. Kannwischer, and Bo-Yin Yang. Multi-moduli NTTs for Saber on Cortex-M3 and Cortex-M4. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(1):127–151, 2022.
- [ACD<sup>+</sup>18] Martin R. Albrecht, Benjamin R. Curtis, Amit Deo, Alex Davidson, Rachel Player, Eamonn W. Postlethwaite, Fernando Virdia, and Thomas Wunderer. Estimate all the LWE, NTRU schemes! In Dario Catalano and Roberto De Prisco, editors, *SCN 18: 11th International Conference on Security in Communication Networks*, volume 11035 of *Lecture Notes in Computer Science*, pages 351–367, Amalfi, Italy, September 5–7, 2018. Springer, Cham, Switzerland.
- [ADPS16] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange - A new hope. In Thorsten Holz and Stefan Savage, editors, *USENIX Security 2016: 25th USENIX Security Symposium*, pages 327–343, Austin, TX, USA, August 10–12, 2016. USENIX Association.
- [APS15] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Cryptology ePrint Archive*, Report 2015/046, 2015.
- [BCD<sup>+</sup>16] Joppe W. Bos, Craig Costello, Léo Ducas, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Ananth Raghunathan, and Douglas Stebila. Frodo: Take off the ring! Practical, quantum-secure key exchange from LWE. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1006–1018, Vienna, Austria, October 24–28, 2016. ACM Press.
- [BDK<sup>+</sup>18] Joppe W. Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS - Kyber: A CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy*, pages 353–367, London, United Kingdom, April 24–26, 2018. IEEE Computer Society Press.

- [BPR12] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 719–737, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany.
- [CHK<sup>+</sup>21] Chi-Ming Marvin Chung, Vincent Hwang, Matthias J. Kannwischer, Gregor Seiler, Cheng-Jhih Shih, and Bo-Yin Yang. NTT multiplication for NTT-unfriendly rings. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(2):159–188, 2021.
- [DKRV18] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-LWR based key exchange, CPA-secure encryption and CCA-secure KEM. In Antoine Joux, Abderrahmane Nitaj, and Tajjeeddine Rachidi, editors, *AFRICACRYPT 18: 10th International Conference on Cryptology in Africa*, volume 10831 of *Lecture Notes in Computer Science*, pages 282–305, Marrakesh, Morocco, May 7–9, 2018. Springer, Cham, Switzerland.
- [GJ21] Qian Guo and Thomas Johansson. Faster dual lattice attacks for solving LWE with applications to CRYSTALS. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part IV*, volume 13093 of *Lecture Notes in Computer Science*, pages 33–62, Singapore, December 6–10, 2021. Springer, Cham, Switzerland.
- [HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017: 15th Theory of Cryptography Conference, Part I*, volume 10677 of *Lecture Notes in Computer Science*, pages 341–371, Baltimore, MD, USA, November 12–15, 2017. Springer, Cham, Switzerland.
- [JZC<sup>+</sup>18] Haodong Jiang, Zhenfeng Zhang, Long Chen, Hong Wang, and Zhi Ma. IND-CCA-secure key encapsulation mechanism in the quantum random oracle model, revisited. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 96–125, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
- [LLL24] Shanxiang Lyu, Ling Liu, and Cong Ling. Learning with quantization: Construction, hardness, and applications. *Cryptology ePrint Archive*, Report 2024/714, 2024.
- [LLZ<sup>+</sup>18] Xianhui Lu, Yamin Liu, Zhenfei Zhang, Dingding Jia, Haiyang Xue, Jingnan He, Bao Li, and Kunpeng Wang. LAC: Practical ring-LWE based public-key encryption with byte-level modulus. *Cryptology ePrint Archive*, Report 2018/1009, 2018.
- [LS15] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.
- [LZL<sup>+</sup>25] Yijian Liu, Yu Zhang, Xianhui Lu, Yao Cheng, and Yongjian Yin. DAWN: Smaller and faster NTRU encryption via double encoding. In *Advances in Cryptology – ASIACRYPT 2025, Part III*, *Lecture Notes in Computer Science*, pages 396–427. Springer, Singapore, Singapore, December 7–11, 2025.
- [MAT] IDF MATZOV. Report on the security of lwe: improved dual lattice attack, 2022. *URL: <https://zenodo.org/record/6412487>*.
- [MKKV21] Jose Maria Bermudo Mera, Angshuman Karmakar, Suparna Kundu, and Ingrid Verbauwhede. Scabbard: a suite of efficient learning with rounding key-encapsulation mechanisms. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(4):474–509, 2021.

- [MKV20] Jose Maria Bermudo Mera, Angshuman Karmakar, and Ingrid Verbauwhede. Time-memory trade-off in Toom-Cook multiplication. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2020(2):222–244, 2020.
- [WSM<sup>+</sup>25] Emily Wenger, Eshika Saxena, Mohamed Malhou, Ellie Thieu, and Kristin E. Lauter. Benchmarking attacks on learning with errors. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 279–297, San Francisco, CA, USA, May 12–15, 2025. IEEE Computer Society Press.
- [WZZ<sup>+</sup>24] Anyu Wang, Zhongxiang Zheng, Chunhuan Zhao, Zhiyuan Qiu, Guang Zeng, Ye Yuan, Changchun Mu, and Xiaoyun Wang. Scloud+: An efficient lwe-based kem without ring/-module structure. In *International Conference on Research in Security Standardisation*, pages 147–174. Springer, 2024.
- [Zam14] Ram Zamir. *Lattice coding for signals and networks: A structured coding approach to quantization, modulation, and multiuser information theory*. Cambridge University Press, 2014.
- [ZFY23] Jiang Zhang, Dengguo Feng, and Di Yan. Nev: faster and smaller ntru encryption using vector decoding. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 157–189. Springer, 2023.
- [ZYF<sup>+</sup>20] Jiang Zhang, Yu Yu, Shuqin Fan, Zhenfeng Zhang, and Kang Yang. Tweaking the asymmetry of asymmetric-key cryptography on lattices: Kems and signatures of smaller sizes. In *IACR International Conference on Public-Key Cryptography*, pages 37–65. Springer, 2020.